VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY

# Komponenta výukového serveru TI - Amortizovaná složitost 1

Component of Teaching Server for Theoretical Computer Science - Amortized Complexity 1

Martin Dressler

Bakalářská práce

Vedoucí práce: Ing. Martin Kot, Ph.D.

Ostrava, 2026

VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Č.j.VSB/25/116928
vsboee25037F9C

# Zadání bakalářské práce

Student: **Martin Dressler**

Studijní program: B0613A140014 Informatika

Téma: Komponenta výukového serveru TI - Amortizovaná složitost 1

Component of Teaching Server for Theoretical Computer Science - Amortized Complexity 1

Jazyk vypracování: čeština

Zásady pro vypracování:

V rámci diplomových a bakalářských prací vzniká výukový server pro předměty teoretické informatiky. Jedná se o sadu dynamických webových stránek umožňujících studentům pochopení různých typů úloh a problémů. Na rozdíl od běžných výukových textů s pevně daným počtem ukázkových příkladů umí tyto stránky generovat libovolně mnoho ukázek na základě vstupů od uživatele. Cílem této konkrétní bakalářské práce je vytvořit komponentu pro pochopení počítání složitosti algoritmů s využitím tzv. amortizované složitosti.

Vytvořte dynamické webové stránky umožňující uživateli následující:
1. Zobrazit si vhodným způsobem simulaci běhu alespoň 3 různých algoritmů, u kterých je vhodné složitost analyzovat pomocí amortizované složitosti.
2. Vstupy těchto algoritmů bude moci uživatel zadávat třemi způsoby:
a) Vhodným, uživatelsky přívětivým, způsobem ručně.
b) Nechat si vstup vygenerovat zcela náhodně podle nastavených parametrů.
c) Nechat si vygenerovat různě velké ukázkové vstupy (typicky např. nejhorší případ, nejlepší případ apod.).
3. Při simulaci algoritmů se bude uživateli zobrazovat:
a) Informace vystihující aktuální stav výpočtu algoritmu (např. obsahy různých datových struktur atd.).
b) Počet provedených kroků.
c) Aktuální hodnotu potenciálu (při použití tzv. potenciálové metody) nebo naspořené mince (při použití tzv. účetní metody).
4. Ke každému algoritmu bude přímo na vytvořené stránce také k zobrazení samotné odvození složitosti tohoto algoritmu (např. formou statické stránky).

Studenti řešící toto zadání s rozdílným číslem v názvu mohou (ale nemusí) spolupracovat tak, že výsledek může mít společné uživatelské rozhraní apod. Ale každý bude implementovat jiné 3 algoritmy.

Seznam doporučené odborné literatury:

[1] Tarjan, Robert Endre: "Amortized Computational Complexity", SIAM Journal on Algebraic and Discrete Methods, 6 (2): 306–318, doi:10.1137/0606031
[2] Sawa, Zdeněk: "Teoretická informatika", podklady pro přednášky, VŠB - Technická univerzita Ostrava, dostupné z URL: https://www.cs.vsb.cz/sawa/ti/slides/ti-slides-03.pdf

[3] Kozen, Dexter: "CS 3110 Lecture 20: Amortized Analysis", Cornell University, dostupné z URL: http://www.cs.cornell.edu/courses/cs3110/2011sp/lectures/lec20-amortized/amortized.htm

Formální náležitosti a rozsah bakalářské práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí bakalářské práce: **Ing. Martin Kot, Ph.D.**

Datum zadání: 01.09.2024

Datum odevzdání: 30.04.2026

Garant studijního programu: Ing. Tomáš Fabián, Ph.D.

V IS EDISON zadáno: 18.08.2025 09:41:44

## Abstrakt

Tato bakalářská práce se zabývá problematikou amortizované složitosti. Cílem praktické části bylo vytvořit komponentu výukového serveru pro předmět Teoretická informatika v navazujícím magisterském studiu. Výsledná aplikace má podobu dynamické webové stránky, která vizualizuje vybrané algoritmy vhodné pro analýzu amortizované složitosti a umožňuje uživateli pracovat s různými typy vstupů. Součástí aplikace je zobrazení jednotlivých kroků algoritmů, změn stavů datových struktur a dalších hodnot důležitých pro pochopení jejich chování. Teoretická část se věnuje vysvětlení amortizované složitosti, souvisejících pojmů a metod, popisu implementace vytvořené komponenty a charakteristice vybraných algoritmů. Výsledkem práce je výukový nástroj, který může přispět k lepšímu pochopení amortizované složitosti studenty.

## Klíčová slova

Amortizovaná složitost, potenciální metoda, vizualizace algoritmů, výukový server, teoretická informatika, webová aplikace, HTML5, CSS3, JavaScript, Bootstrap

## Abstract

This bachelor's thesis deals with the issue of amortized complexity. The goal of the practical part was to create a component of a teaching server for the course Theoretical Computer Science in the follow-up master's degree programme. The resulting application takes the form of a dynamic web page that visualizes selected algorithms suitable for amortized complexity analysis and allows the user to work with different types of inputs. The application includes the display of individual algorithm steps, changes in the states of data structures, and other values important for understanding their behavior. The theoretical part focuses on the explanation of amortized complexity, related concepts and methods, the description of the implementation of the created component, and the characteristics of the selected algorithms. The result of the thesis is an educational tool that can contribute to a better understanding of amortized complexity by students.

## Keywords

Amortized complexity, potential method, algorithm visualization, teaching server, theoretical computer science, web application, HTML5, CSS3, JavaScript, Bootstrap

## Poděkování

Rád bych poděkoval vedoucímu mé bakalářské práce Ing. Martinu Kotovi, Ph.D. za odborné vedení, cenné rady, trpělivost a čas, který mi věnoval při zpracování této práce. Zároveň mu děkuji za možnost věnovat se tématu, které může přispět k lepšímu pochopení amortizované složitosti budoucím studentům.

# Obsah

**Seznam použitých symbolů a zkratek** **9**

**Seznam obrázků** **10**

**Seznam tabulek** **11**

**1 Úvod** **12**

**2 Základní problematika** **13**

2.1 Základní pojmy analýzy algoritmů . . . . . 13
2.2 Časová složitost algoritmů . . . . . 14
2.2.1 Velikost vstupu a definice časové složitosti . . . . . 14
2.2.2 Časová složitost v nejhorším a průměrném případě . . . . . 15
2.2.3 Asymptotická notace a třídy růstu funkcí . . . . . 17
2.2.4 Praktický význam časové složitosti . . . . . 18
2.3 Amortizovaná složitost a její význam . . . . . 19
2.3.1 Vymezení a význam amortizované analýzy . . . . . 19
2.3.2 Srovnání s ostatními přístupy . . . . . 20
2.4 Metody amortizované analýzy . . . . . 21
2.4.1 Agregovaná metoda . . . . . 21
2.4.2 Účetní metoda . . . . . 21
2.4.3 Potenciálová metoda . . . . . 21
2.4.4 Vztah mezi jednotlivými metodami . . . . . 22
2.5 Potenciálová metoda . . . . . 22
2.5.1 Princip potenciálové metody . . . . . 22
2.5.2 Potenciálová funkce a amortizovaná cena operace . . . . . 23
2.5.3 Význam a využití potenciálové metody . . . . . 24
2.6 Vybrané algoritmy vhodné pro amortizovanou analýzu . . . . . 24

**3 Návrh komponenty výukového serveru** **26**
3.1 Cíl a role navrhované komponenty . . . 26
3.1.1 Zařazení komponenty do výukového serveru . . . 26
3.1.2 Hlavní účel komponenty . . . 27
3.1.3 Přínos komponenty pro výuku amortizované složitosti . . . 27
3.2 Požadavky na komponentu . . . 27
3.2.1 Funkční požadavky . . . 28
3.2.2 Požadavky na uživatelskou interakci, názornost a výukovou hodnotu . . . 29
3.3 Výběr algoritmů pro praktickou část . . . 29
3.3.1 Kritéria výběru algoritmů . . . 30
3.3.2 Multipop na zásobníku jako jednoduchý příklad . . . 30
3.3.3 Fronta využívající dva zásobníky jako středně obtížný příklad . . . 30
3.3.4 Splay strom jako pokročilejší příklad . . . 31
3.4 Návrh funkcionality komponenty . . . 31
3.4.1 Režimy zadávání vstupů . . . 32
3.4.2 Zobrazení průběhu algoritmu po jednotlivých krocích . . . 34
3.4.3 Zobrazení počtu kroků, počtu operací a potenciálu . . . 35
3.4.4 Zobrazení doplňujících informací o aktuálním stavu algoritmu . . . 36
3.5 Návrh uživatelského rozhraní . . . 38
3.5.1 Návrh hlavní stránky a výběru algoritmu . . . 38
3.5.2 Návrh ovládacích prvků simulace . . . 39
3.5.3 Návrh vizualizace datových struktur . . . 40
3.5.4 Návrh informačních panelů a doprovodných vysvětlení o algoritmech . . . 42
3.6 UML komponentový diagram aplikace . . . 43
3.7 Návrh struktury komponenty . . . 45

**4 Implementace komponenty** **46**
4.1 Použité technologie a vývojové prostředí . . . 46
4.2 Struktura a realizace aplikace . . . 47
4.2.1 Organizace zdrojových souborů . . . 47
4.2.2 Hlavní části aplikační logiky . . . 48
4.2.3 Implementace vybraných algoritmů . . . 50
4.2.4 Zpracování vstupů a režimů spuštění . . . 53
4.3 Uživatelské rozhraní a responzivita . . . 54
4.3.1 Struktura uživatelského rozhraní a práce s dynamickým obsahem . . . 54
4.3.2 Responzivní zobrazení na různých zařízeních . . . 56
4.4 Nasazení a zpřístupnění aplikace . . . 57

**5 Závěr** **58**

# Seznam použitých zkratek a symbolů

| | | |
|---|---|---|
| TI | – | Teoretická informatika |
| API | – | Application Programming Interface |
| UI | – | User Interface |
| DOM | – | Document Object Model |
| HTML | – | HyperText Markup Language |
| CSS | – | Cascading Style Sheets |
| JS | – | JavaScript |
| LIFO | – | Last In, First Out |
| FIFO | – | First In, First Out |
| BST | – | Binary Search Tree |
| PDF | – | Portable Document Format |
| URL | – | Uniform Resource Locator |
| HTTP | – | HyperText Transfer Protocol |
| UML | – | Unified Modeling Language |

# Seznam obrázků

2.2.1 Časová složitost v nejhorším případě . . . . . 16
2.2.2 Časová složitost v průměrném případě . . . . . 17

3.4.1 Hlavní nabídka režimů zadávání vstupů u zvoleného algoritmu . . . . . 32
3.4.2 Příklady jednotlivých režimů zadávání vstupů . . . . . 33
3.4.3 Zobrazení průběhu algoritmu po jednotlivých krocích v režimu syntaxe . . . . . 34
3.4.4 Automatický průběh simulace v režimu vybraného typického případu . . . . . 35
3.4.5 Zobrazení počtu kroků, počtu operací a aktuální hodnoty potenciálu v průběhu simulace 36
3.4.6 Detailní doplňující informace dostupné v historii provedených kroků . . . . . 37
3.4.7 Doplňující informační panel zobrazující aktuální stav operace při práci s algoritmem 37
3.5.1 Hlavní stránka komponenty s výběrem algoritmů, přepínačem jazykových mutací a tlačítkem pro informace o bakalářské práci . . . . . 39
3.5.2 Příklad ovládacích prvků simulace v režimu syntaxe . . . . . 40
3.5.3 Příklady vizualizace datových struktur použitých v navrhované komponentě . . . . . 41
3.5.4 Příklad informačního panelu s doprovodným vysvětlením vybraného algoritmu . . . 42
3.6.1 UML komponentový diagram aplikace . . . . . 44

4.1 Ukázka hlavní aplikační logiky ve funkci changeLanguage() . . . . . 49
4.2 Zkrácená ukázka implementace operace multipop . . . . . 50
4.3 Zkrácená ukázka přesunu prvků při implementaci fronty pomocí dvou zásobníků . . 52
4.4 Zkrácená ukázka levé rotace ve splay stromu . . . . . 52
4.5 Zkrácená ukázka konfigurace vstupních režimů . . . . . 53
4.6 Zkrácená ukázka struktury uživatelského rozhraní v souboru index.html . . . . . 55
4.3.1 Porovnání zobrazení hlavní stránky na různých typech zařízení . . . . . 56
4.7 Zkrácená ukázka responzivních pravidel pro mobilní zařízení . . . . . 57

# Seznam tabulek

2.1.1 Základní pojmy používané při analýze algoritmů . . . . . 14
2.3.1 Srovnání vybraných přístupů k hodnocení složitosti . . . . . 20
2.4.1 Základní metody amortizované analýzy . . . . . 22
2.6.1 Vybrané algoritmy vhodné pro amortizovanou analýzu . . . . . 25

3.2.1 Hlavní funkční požadavky navrhované komponenty . . . . . 28

# Kapitola 1

# Úvod

Hlavním cílem této bakalářské práce bylo vytvoření komponenty pro výukový server v oblasti teoretické informatiky. V praktické části bylo potřeba navrhnout a implementovat dynamickou webovou aplikaci, která by sloužila jako názorný a interaktivní nástroj pro pochopení vybraných algoritmů a principů amortizované složitosti. Výsledná aplikace má studentům umožnit sledovat průběh jednotlivých operací, změny stavů datových struktur a další důležité hodnoty potřebné pro lepší pochopení chování algoritmů i jejich amortizované složitosti.

Problematika amortizované složitosti patří mezi velmi důležitá témata při studiu algoritmů a datových struktur. Její význam spočívá v tom, že umožňuje lépe popsat chování algoritmů v situacích, kdy jednotlivé operace nemají vždy stejnou cenu a jejich skutečná náročnost se projeví až v rámci delší posloupnosti kroků. U některých algoritmů a datových struktur nestačí hodnotit jednotlivé operace samostatně, protože jejich skutečná efektivita se ukáže teprve při sledování delší posloupnosti operací.

I když je amortizovaná složitost významnou součástí TI, bývá pro mnoho studentů obtížně pochopitelná. Hlavním důvodem je její abstraktnější charakter, protože při snaze o její pochopení nestačí sledovat pouze výsledek jedné operace, ale je také nutné porozumět souvislostem mezi více operacemi, změnám stavů datových struktur a principům, podle nichž se náklady mezi jednotlivými kroky rozkládají. Z tohoto důvodu byla tato bakalářská práce vytvořena jako podpora teoretického výkladu prostřednictvím interaktivní vizualizace a možnosti sledovat chování algoritmů na různých příkladech.

Výsledkem této práce je dynamická webová aplikace, která může sloužit budoucím studentům jako výuková pomůcka pro názornou vizualizaci vybraných algoritmů. Důraz je kladen především na možnost aktivní práce uživatele se vstupy a na sledování chování algoritmů v jednotlivých krocích. Student tak může lépe porozumět tomu, jak se mění stav datové struktury, jak se vyvíjí průběh operací a proč nelze u některých algoritmů posuzovat jejich efektivitu pouze na základě jedné samostatné operace. Výsledná komponenta má za cíl propojit teoretický výklad s praktickou ukázkou a přispět k názornějšímu pochopení celé problematiky.

# Kapitola 2

# Základní problematika

Amortizovaná složitost představuje způsob hodnocení algoritmů a datových struktur, u nichž se náklady jednotlivých operací mohou výrazně lišit. Pro její správné pochopení je nejprve nutné vycházet ze základních pojmů analýzy algoritmů a z běžného chápání časové složitosti. Teprve na tomto základě lze lépe vysvětlit, v čem se amortizovaná analýza liší od jiných přístupů a proč je pro některé typy algoritmů vhodná.

## 2.1 Základní pojmy analýzy algoritmů

Analýza algoritmů spočívá ve zkoumání vlastností algoritmů, především jejich správnosti a náročnosti výpočtu. V běžné praxi bývá přesné vyjádření složitosti obtížné, protože závisí na konkrétním výpočetním modelu i na konkrétní implementaci algoritmu. Z tohoto důvodu se v teorii algoritmů využívá asymptotické vyjadřování, které potlačuje méně podstatné detaily a umožňuje soustředit se především na to, jak rychle roste náročnost algoritmu se zvětšující se velikostí vstupu [1].

Mezi nejzákladnější pojmy, které se v analýze algoritmů běžně používají, patří algoritmus, vstup, výstup, korektnost, časová složitost, paměťová složitost a asymptotické vyjádření. Jejich stručnou charakteristiku shrnuje tabulka 2.1.1, která současně vytváří základ pro další výklad časové složitosti i amortizované analýzy.

Tabulka 2.1.1: Základní pojmy používané při analýze algoritmů

| **Pojem** | **Stručné vysvětlení** |
|---|---|
| Algoritmus | Jednoznačně definovaný postup, který převádí vstupní data na požadovaný výstup. |
| Vstup | Data, nad nimiž je algoritmus vykonáván. Velikost vstupu obvykle ovlivňuje náročnost výpočtu. |
| Výstup | Výsledek zpracování vstupu algoritmem. |
| Korektnost | Vlastnost algoritmu, která určuje, zda algoritmus poskytuje správný výsledek pro přípustné vstupy. |
| Časová složitost | Vyjadřuje náročnost algoritmu z hlediska času, resp. počtu provedených operací. |
| Paměťová složitost | Vyjadřuje nároky algoritmu na paměť během výpočtu. |
| Asymptotické vyjádření | Způsob popisu růstu náročnosti algoritmu pro velké vstupy bez zdůrazňování méně podstatných detailů. |

## 2.2 Časová složitost algoritmů

Časová složitost patří mezi základní pojmy analýzy algoritmů a slouží k porovnávání jejich výpočetní náročnosti. Umožňuje popsat, jak se mění počet provedených operací v závislosti na velikosti vstupu, a vytváří tak důležitý základ pro další úvahy o efektivitě algoritmů i pro pochopení amortizované analýzy.

### 2.2.1 Velikost vstupu a definice časové složitosti

Časová složitost algoritmů patří mezi nejdůležitější charakteristiky používané při hodnocení efektivity algoritmických řešení. Počítače sice pracují velmi rychle, jejich výpočetní možnosti však nejsou neomezené, a proto i provedení jednotlivých instrukcí vyžaduje určitý čas. Stejný problém navíc může být řešen více různými algoritmy, jejichž doba výpočtu se může výrazně lišit. Z tohoto důvodu je vhodné algoritmy mezi sebou porovnávat a zkoumat jejich náročnost z hlediska počtu provedených operací.

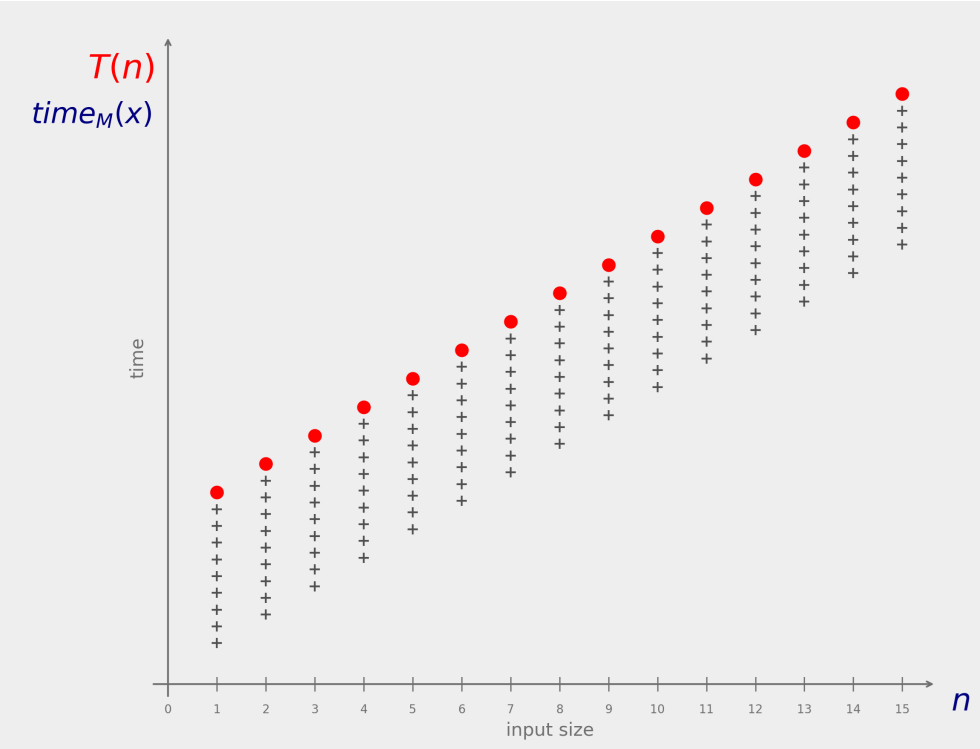
Při běžném programování je sice možné algoritmus implementovat a následně změřit dobu jeho běhu na konkrétních vstupních datech, takový přístup však neposkytuje dostatečně obecnou informaci o chování algoritmu pro všechny možné vstupy. V teorii algoritmů se proto časová složitost nevyjadřuje přímo pomocí skutečného času naměřeného na konkrétním zařízení, ale spíše pomocí počtu provedených instrukcí nebo elementárních operací. Díky tomu je možné odhlédnout od konkrétního hardwaru a zaměřit se na samotné chování algoritmu.

Důležitým pojmem při analýze časové složitosti je velikost vstupu. Pro různé typy problémů může být definována různě. Například u problému třídění lze za velikost vstupu považovat počet prvků ve vstupní posloupnosti, u problémů pracujících s čísly může být přirozenější zvolit počet bitů potřebných k jejich zápisu a u grafových problémů lze velikost vstupu vyjádřit například dvojicí hodnot udávajících počet vrcholů a počet hran. Zvolená definice velikosti vstupu tedy není vždy pevně dána, ale měla by přirozeně odpovídat povaze analyzovaného problému.

Uvažujme nyní konkrétní algoritmus a jeho implementaci. Je přirozené očekávat, že pro vstupy různé velikosti bude obvykle potřeba různý počet instrukcí. Ani pro vstupy stejné velikosti však nemusí algoritmus vždy vykonat stejný počet instrukcí, protože jeho chování může záviset na konkrétním uspořádání vstupních dat. Proto se v teorii často zavádí funkce $T(n)$, která pro každou velikost vstupu $n$ udává maximální počet instrukcí provedených algoritmem na vstupu velikosti $n$. Takto definovaná funkce představuje časovou složitost algoritmu v nejhorším případě a umožňuje zachytit horní mez výpočetní náročnosti [1].

### 2.2.2 Časová složitost v nejhorším a průměrném případě

Pro lepší názornost je princip časové složitosti v nejhorším případě znázorněn na obrázku 2.2.1. Jednotlivé body představují různé vstupy stejné nebo podobné velikosti a jejich skutečnou časovou náročnost. Funkce $T(n)$ zde odpovídá horní hranici těchto hodnot, tedy nejhoršímu případu pro danou velikost vstupu.

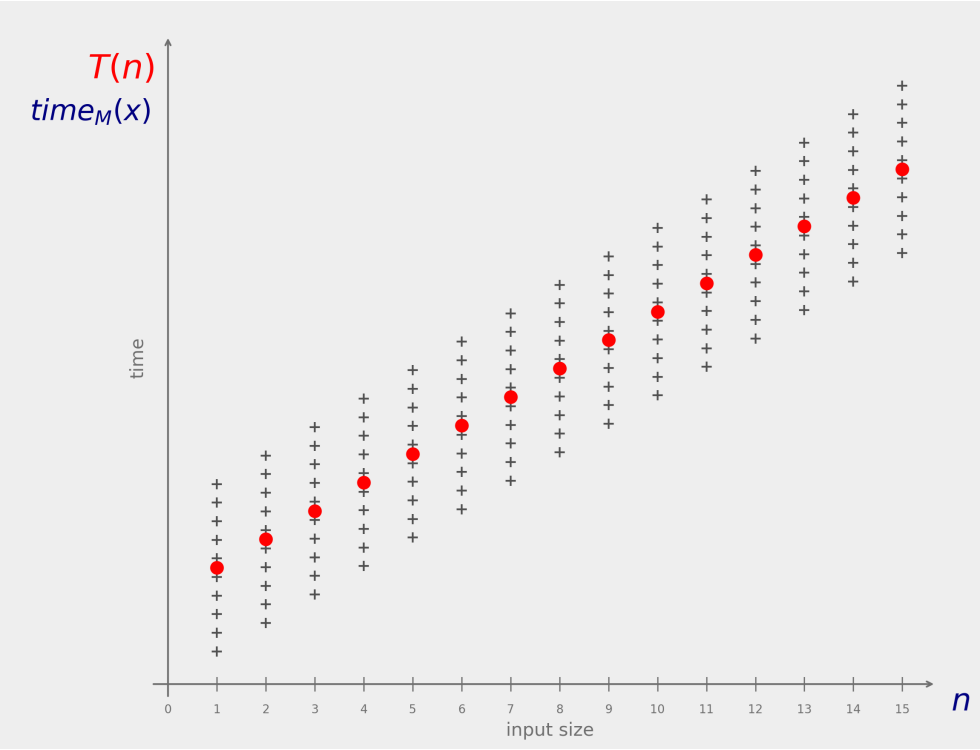



Obrázek 2.2.1: Časová složitost v nejhorším případě

Obrázek 2.2.1 ukazuje, že pro vstupy stejné velikosti nemusí algoritmus vždy vykonat stejný počet operací. Časová složitost v nejhorším případě proto sleduje maximální hodnotu těchto nákladů a poskytuje tak bezpečný odhad horní meze časové náročnosti algoritmu.

Vedle časové složitosti v nejhorším případě má smysl zkoumat také časovou složitost v průměrném případě. V tomto případě není funkce $T(n)$ určena maximální hodnotou přes všechny vstupy dané velikosti, ale aritmetickým průměrem odpovídajících nákladů. Určení průměrného případu však bývá zpravidla obtížnější než určení nejhoršího případu, protože vyžaduje přesnější představu o množině vstupů a jejich rozdělení. Ve většině případů se proto při analýze algoritmů častěji vychází právě z nejhoršího případu.

Pro názornost je na obrázku 2.2.2 znázorněna časová složitost v průměrném případě. Na rozdíl od nejhoršího případu zde funkce $T(n)$ nepředstavuje horní obálku všech hodnot, ale jejich průměrné chování pro vstupy stejné velikosti.




Obrázek 2.2.2: Časová složitost v průměrném případě

Časová složitost v nejhorším a průměrném případě se často příliš neliší, i když v některých situacích může být rozdíl významný. Zároveň se uvádí, že zkoumání složitosti v nejlepším případě obvykle nemá příliš velký praktický význam.

### 2.2.3 Asymptotická notace a třídy růstu funkcí

Přesné vyjádření časové složitosti bývá často komplikované a silně závisí na zvoleném výpočetním modelu i na detailech implementace. Ve většině případů navíc není nutné znát přesný počet instrukcí, ale stačí odhadnout, jak rychle tento počet roste se zvětšující se velikostí vstupu. Právě z tohoto důvodu se zavádí asymptotická notace, která zanedbává méně podstatné detaily a umožňuje soustředit se na dominantní složku růstu sledované funkce.

V rámci asymptotické notace se nejčastěji používají symboly $O$, $\Omega$ a $\Theta$. Zápis $O(g(n))$ vyjadřuje horní asymptotickou mez růstu funkce, tedy skutečnost, že sledovaná funkce neroste rychleji než funkce $g(n)$ až na konstantní násobek. Naopak $\Omega(g(n))$ vyjadřuje dolní asymptotickou mez a $\Theta(g(n))$ charakterizuje situaci, kdy funkce roste stejnou rychlostí jako $g(n)$ z hlediska horní i dolní meze

současně. Kromě těchto symbolů se používají také $o(g(n))$ a $\omega(g(n))$, které popisují přísně pomalejší, respektive přísně rychlejší růst [1].

Big-O notace popisuje asymptotické chování funkcí a umožňuje při analýze algoritmů zanedbat konstanty i pomaleji rostoucí členy. Na základě asymptotického chování lze algoritmy rozdělovat do několika základních tříd. Mezi běžné třídy patří například $O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$, polynomiální růst a exponenciální růst. Za logaritmické se považují algoritmy se složitostí $\Theta(\log n)$, za lineární algoritmy se složitostí $\Theta(n)$, za kvadratické se považují algoritmy se složitostí $\Theta(n^2)$ a podobně. Významnou skupinu tvoří také polynomiální algoritmy, tedy algoritmy, jejichž časová složitost je shora omezena nějakým polynomem. Naopak exponenciální nebo faktoriální složitosti obvykle vedou k tomu, že algoritmus je použitelný pouze pro velmi malé vstupy [2].

### 2.2.4 Praktický význam časové složitosti

Při interpretaci asymptotické notace je však nutné mít na paměti i její omezení. Asymptotické odhady vypovídají pouze o tom, jak roste náročnost s rostoucí velikostí vstupu, ale neříkají nic o skutečné době běhu na konkrétním počítači. V notaci mohou být navíc skryty velké konstanty, takže algoritmus s asymptoticky lepší složitostí nemusí být v praxi rychlejší pro malé nebo středně velké vstupy.

Tuto skutečnost lze ilustrovat na třídicích algoritmech. Například quicksort má v nejhorším případě složitost $\Theta(n^2)$, zatímco heapsort $\Theta(n \log n)$, ale přesto bývá quicksort v praxi často velmi rychlý. Z toho vyplývá, že časová složitost je sice velmi důležitým nástrojem pro porovnávání algoritmů, ale při hodnocení konkrétního řešení je třeba zohlednit i další okolnosti.

Z praktického hlediska bývají za efektivní považovány především algoritmy s polynomiální časovou složitostí. Ani zde však nelze formulovat zcela absolutní závěry, protože například algoritmus se složitostí $\Theta(n^{100})$ je sice formálně polynomiální, avšak z praktického hlediska obtížně použitelný. Naopak některé algoritmy s horší teoretickou složitostí mohou být pro malé vstupy nebo specifické úlohy stále využitelné [2].

Pro potřeby této práce je důležité, že klasická časová složitost obvykle hodnotí náklady jednotlivých výpočtů nebo jednotlivých operací vzhledem k velikosti vstupu. U některých datových struktur a algoritmů však může být vhodnější sledovat celkové náklady delší posloupnosti operací. Právě tímto pohledem se zabývá amortizovaná analýza, která je tématem následující části kapitoly.

## 2.3 Amortizovaná složitost a její význam

Amortizovaná složitost patří mezi přístupy, které umožňují přesněji vystihnout chování algoritmů a datových struktur v situacích, kdy se náklady jednotlivých operací mohou výrazně lišit. Její význam spočívá v tom, že neposuzuje každou operaci izolovaně, ale bere v úvahu širší souvislosti v rámci celé posloupnosti operací. Díky tomu může lépe popsat dlouhodobé chování některých algoritmů, než kdyby byly jednotlivé kroky hodnoceny samostatně.

### 2.3.1 Vymezení a význam amortizované analýzy

Amortizovaná složitost představuje způsob hodnocení algoritmů a datových struktur, při němž se nesleduje pouze cena jedné izolované operace, ale celkové náklady delší posloupnosti operací. Tento přístup vychází z toho, že jednotlivé operace v posloupnosti nemusí mít stejnou cenu. Některé mohou být velmi levné, jiné naopak výrazně dražší, a proto by samostatné hodnocení jedné operace mohlo vést ke zkreslenému pohledu na skutečné chování algoritmu [3, 4].

Amortizovaná analýza představuje analýzu posloupnosti operací v nejhorším případě, jejímž cílem je získat těsnější odhad celkových nebo průměrných nákladů na jednu operaci, než kdyby byla každá operace analyzována samostatně [4]. Z toho vyplývá, že dražší operace nejsou posuzovány izolovaně, ale v souvislosti s ostatními operacemi v posloupnosti. Jejich vysoká cena se tak rozkládá mezi více levnějších operací, takže dlouhodobé chování algoritmu může být z hlediska jedné operace příznivější.

Tento princip lze ilustrovat jednoduchým příkladem zásobníku s operacemi *Push*, *Pop* a *Multipop*. Zatímco operace *Multipop* může v jednom okamžiku odebrat více prvků najednou, a být tak dražší než běžné operace, v delší posloupnosti operací každý vložený prvek může být odstraněn pouze omezený početkrát. I když tedy jednotlivá operace může působit nákladně, celkové chování celé posloupnosti zůstává příznivější.

Význam amortizované analýzy spočívá především v tom, že poskytuje realističtější pohled na chování algoritmů a datových struktur v situacích, kdy se náklady jednotlivých operací výrazně liší. Amortizace je chápána jako technika „zprůměrování v čase“ a amortizovaná doba běhu představuje realistickou a zároveň robustní míru složitosti [3]. Tento přístup je užitečný zejména tam, kde se vyskytují občasné drahé operace, které jsou však v delší posloupnosti kompenzovány velkým množstvím operací levných.

Amortizovaná analýza je důležitá také proto, že se nespoléhá na pravděpodobnostní předpoklady o rozdělení vstupů. Oproti průměrnému případu není založena na odhadu, jak často se určitý typ vstupu vyskytne. Hodnocení je prováděno nad posloupností operací v nejhorším případě a může vést k přesnějším odhadům, než kdyby byly jednotlivé operace posuzovány odděleně [4]. Tato vlastnost představuje výhodu zejména u datových struktur, jejichž stav se v čase mění a kde jsou jednotlivé operace vzájemně závislé.

Z hlediska návrhu algoritmů a datových struktur je amortizovaná analýza přínosná i tím, že umožňuje lépe vysvětlit, proč mohou být některá řešení v praxi efektivní, i když obsahují občasné nákladné kroky. Pokud je totiž celková cena dlouhé posloupnosti operací příznivá, může být daná datová struktura nebo algoritmus vhodným řešením i přesto, že jednotlivé operace nemají vždy malou cenu nejhoršího případu.

### 2.3.2 Srovnání s ostatními přístupy

Amortizovanou analýzu je vhodné odlišit od dalších běžných přístupů používaných při hodnocení algoritmů. Základní rozdíly mezi nejhorším případem, průměrným případem a amortizovanou analýzou shrnuje tabulka 2.3.1.

Tabulka 2.3.1: Srovnání vybraných přístupů k hodnocení složitosti

| **Přístup** | **Co sleduje** | **Charakteristika** |
|---|---|---|
| Nejhorší případ | Maximální cenu jedné operace nebo jednoho běhu | Poskytuje bezpečnou horní mez časové náročnosti, ale může působit příliš pesimisticky. |
| Průměrný případ | Očekávanou cenu vzhledem k rozdělení vstupů | Závisí na předpokladech o vstupních datech a jejich pravděpodobnostním rozdělení. |
| Amortizovaná analýza | Celkové náklady posloupnosti operací | Hodnotí dlouhodobé chování celé posloupnosti operací, rozkládá cenu dražších kroků mezi levnější operace a nevyžaduje pravděpodobnostní model vstupů. |

Ze srovnání vyplývá, že amortizovaná analýza zaujímá mezi ostatními přístupy zvláštní postavení. Zachovává výhodu pohledu nejhoršího případu v tom, že neposkytuje pouze optimistický odhad založený na pravděpodobnosti, ale zároveň dovoluje přesněji zachytit skutečné dlouhodobé chování algoritmu. Díky tomu je vhodná pro analýzu datových struktur, u nichž se stav postupně mění a kde některé operace ovlivňují cenu operací následujících.

Na základní vymezení pojmu amortizované složitosti navazují další části této kapitoly, které se podrobněji věnují jednotlivým metodám amortizované analýzy a jejich praktickému použití.

## 2.4 Metody amortizované analýzy

Amortizovanou složitost lze studovat několika různými způsoby, které se liší mírou názornosti i formální přesnosti. Jednotlivé metody přitom nepředstavují vzájemně soupeřící přístupy, ale různé možnosti, jak popsat a odůvodnit chování delších posloupností operací. Jejich znalost je důležitá nejen pro samotnou analýzu algoritmů, ale také pro volbu vhodného způsobu výkladu při objasňování amortizované složitosti.

### 2.4.1 Agregovaná metoda

Agregovaná metoda představuje nejjednodušší způsob amortizované analýzy. Jejím základním principem je sledování celkových nákladů celé posloupnosti operací a následné odvození průměrných nákladů připadajících na jednu operaci. Místo samostatného hodnocení jednotlivých operací se tedy analyzuje celkový součet jejich cen. Pokud lze ukázat, že posloupnost $m$ operací má celkovou cenu $O(m)$, je možné usoudit, že amortizovaná cena jedné operace je konstantní [5].

Výhodou agregované metody je její jednoduchost a přehlednost. Hodí se zejména v situacích, kdy lze celkové náklady celé posloupnosti vyjádřit přímo a bez zavádění dalších pomocných veličin. Naopak u složitějších datových struktur bývá tato metoda méně flexibilní.

### 2.4.2 Účetní metoda

Účetní metoda, označovaná také jako bankéřská metoda, vychází z představy, že jednotlivým operacím lze přiřadit určitou účtovanou cenu, která se může lišit od jejich skutečné ceny. Levnějším operacím je přiřazena vyšší cena, než jakou ve skutečnosti mají, a vzniklý přebytek se uchovává jako rezerva pro budoucí dražší operace. Díky tomu je možné rozložit cenu nákladnějších kroků mezi větší počet kroků, které jsou levnější [5].

U této metody je důležité, aby nahromaděná rezerva v průběhu celé posloupnosti operací nikdy neklesla pod nulu. Účetní metoda tak názorně ukazuje, že některé operace si mohou vytvářet rezervu pro budoucí nákladnější kroky.

### 2.4.3 Potenciálová metoda

Potenciálová metoda pracuje s potenciálovou funkcí, která vyjadřuje stav datové struktury a určitou rezervu nahromaděnou v systému. Amortizovaná cena operace je zde určena jako součet její skutečné ceny a změny potenciálu mezi stavem před operací a po ní [5]. Vzhledem k její obecnosti a významu bude tato metoda podrobněji vysvětlena v následující části kapitoly.

### 2.4.4 Vztah mezi jednotlivými metodami

Agregovaná metoda, účetní metoda a potenciálová metoda představují tři základní způsoby, jak přistupovat k amortizované analýze. Jejich hlavní vlastnosti a rozdíly shrnuje tabulka 2.4.1.

Tabulka 2.4.1: Základní metody amortizované analýzy

| **Metoda** | **Základní princip** | **Charakteristika** |
|---|---|---|
| Agregovaná metoda | Sleduje celkovou cenu celé posloupnosti operací | Nejjednodušší přístup, vhodný zejména tehdy, když lze celkové náklady celé posloupnosti přímo a přehledně vyjádřit. |
| Účetní metoda | Přiřazuje operacím účtované ceny a vytváří rezervu pro dražší operace | Názorný přístup založený na představě „ukládání rezervy“, která se později využívá při nákladnějších krocích. |
| Potenciálová metoda | Pracuje s potenciálovou funkcí vyjadřující stav datové struktury | Obecný a formální přístup, který umožňuje přesně zachytit vztah mezi změnou stavu struktury a amortizovanou cenou operace. |

Z uvedeného srovnání vyplývá, že jednotlivé metody nejsou ve vzájemném rozporu, ale spíše představují různé pohledy na tentýž problém. Zatímco agregovaná metoda bývá nejjednodušší na použití, potenciálová metoda nabízí největší obecnost a formální sílu. Právě proto je v další části této kapitoly podrobněji rozebrána.

## 2.5 Potenciálová metoda

Potenciálová metoda představuje jeden z nejdůležitějších nástrojů amortizované analýzy, protože umožňuje formálně popsat vztah mezi aktuálním stavem datové struktury a náklady budoucích operací. Její význam spočívá v tom, že dokáže zachytit nejen cenu právě prováděného kroku, ale i to, jak se změnou stavu systému vytváří nebo spotřebovává rezerva pro další operace.

### 2.5.1 Princip potenciálové metody

Potenciálová metoda patří mezi základní metody amortizované analýzy a představuje obecný způsob, jak vyjádřit vztah mezi skutečnými náklady jednotlivých operací a jejich dlouhodobým chováním v celé posloupnosti operací. Vychází z předpokladu, že aktuální stav datové struktury může

nést určitou rezervu pro budoucí nákladnější operace. Tato rezerva je popsána pomocí potenciálové funkce [3].

Potenciálová metoda proto nesleduje pouze skutečnou cenu právě prováděné operace, ale také to, jak se po jejím provedení změní stav celé datové struktury. Díky tomu umožňuje přesněji zachytit celkové chování delších posloupností operací než izolované hodnocení jednotlivých kroků. Ve srovnání s agregovanou a účetní metodou je abstraktnější, ale zároveň poskytuje obecný a formální nástroj, který je vhodný i pro složitější důkazy amortizované složitosti.

### 2.5.2 Potenciálová funkce a amortizovaná cena operace

Klíčovým pojmem této metody je potenciálová funkce, obvykle značená $\Phi$. Tato funkce přiřazuje každému stavu datové struktury určitou číselnou hodnotu, která vyjadřuje velikost nahromaděné rezervy v daném stavu. Potenciál tedy nepředstavuje skutečný čas ani skutečný počet instrukcí, ale pomocnou veličinu, která umožňuje lépe vyjádřit rozložení nákladů mezi jednotlivé operace.

Je-li skutečná cena $i$-té operace označena jako $c_i$, stav datové struktury před provedením operace jako $D_{i-1}$ a stav po jejím provedení jako $D_i$, pak amortizovaná cena této operace může být zapsána ve tvaru

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

Tento vztah vyjadřuje, že amortizovaná cena operace se skládá ze skutečné ceny operace a změny potenciálu mezi stavem před operací a po ní. Pokud se potenciál po operaci zvětší, znamená to, že se vytváří rezerva pro budoucí operace. Pokud se naopak potenciál zmenší, dochází k využití dříve nahromaděné rezervy.

Pro celou posloupnost $m$ operací pak platí

$$\sum_{i=1}^{m} \hat{c}_i = \sum_{i=1}^{m} c_i + \Phi(D_m) - \Phi(D_0).$$

Z tohoto vztahu je patrné, že se při sečtení amortizovaných cen většina změn potenciálu navzájem vyruší a výsledkem zůstane součet skutečných cen operací upravený pouze o rozdíl koncového a počátečního potenciálu. Důležitou vlastností potenciálové funkce je proto to, že by měla být zvolena tak, aby její hodnoty byly nezáporné, případně aby počáteční stav neměl záporný potenciál. Díky tomu lze ukázat, že součet amortizovaných cen celé posloupnosti poskytuje horní odhad součtu jejich skutečných cen. Potenciálová metoda tak umožňuje dokázat, že i když některé operace mohou být drahé, jejich cena je v delší posloupnosti vyvážena operacemi jinými.

Z praktického hlediska je při volbě potenciálové funkce důležité, aby odpovídala vlastnostem konkrétní datové struktury a správně vystihovala, jakým způsobem systém vytváří a spotřebovává rezervu. Právě volba vhodné potenciálové funkce bývá při důkazu amortizované složitosti jedním z nejdůležitějších kroků.

### 2.5.3 Význam a využití potenciálové metody

Význam potenciálové metody spočívá především v její obecnosti a formální síle. Ve srovnání s ostatními metodami amortizované analýzy umožňuje velmi přesně popsat změny stavu datové struktury a jejich vliv na cenu budoucích operací. Díky tomu je vhodná i pro analýzu složitějších struktur, u nichž nestačí pouze sledovat celkový součet nákladů nebo intuitivně rozdělovat rezervu mezi jednotlivé operace.

Potenciálová metoda je užitečná zejména v situacích, kdy se stav datové struktury v čase výrazně mění a kdy některé operace přímo ovlivňují náročnost operací následujících. V takovém případě poskytuje přirozený způsob, jak vyjádřit, že nákladnější operace nejsou izolovaným jevem, ale souvisejí s vývojem celého systému. To je také důvod, proč je tato metoda často považována za nejuniverzálnější z běžně používaných metod amortizované analýzy.

Z hlediska této práce je potenciálová metoda důležitá i proto, že umožňuje velmi přehledně sledovat, jak se v jednotlivých stavech vytváří nebo spotřebovává rezerva, a tím názorně vysvětlit, proč nelze cenu některých algoritmů hodnotit pouze podle jedné samostatné operace. Právě tato vlastnost činí potenciálovou metodu vhodnou nejen pro teoretickou analýzu, ale i pro výukové a vizualizační účely.

## 2.6 Vybrané algoritmy vhodné pro amortizovanou analýzu

Amortizovaná analýza se uplatňuje zejména u algoritmů, u nichž se jednotlivé operace mohou lišit svou skutečnou cenou, avšak při delší posloupnosti operací vykazuje celý systém příznivější chování. Tento přístup je vhodný především tehdy, když se vyskytují občasné nákladnější operace, jejichž cenu lze rozložit mezi větší počet operací levnějších. Typickou situací je stav, kdy se mění datová struktura a některé operace ovlivňují náročnost operací následujících.

Mezi často uváděné příklady algoritmů vhodných pro amortizovanou analýzu patří například:

- Multipop na zásobníku,
- Fronta využívající dva zásobníky,
- Splay strom,
- Dynamické pole s občasným rozšířením kapacity,
- Binární čítač,
- některé operace nad disjoint-set strukturami.

Tyto algoritmy mají společné to, že jednotlivé operace nemusí mít vždy stejnou cenu, ale v delší posloupnosti operací lze jejich chování pomocí amortizované analýzy popsat přesněji, než kdyby bylo hodnoceno pouze izolovaně po jednotlivých krocích.

Do praktické části této bakalářské práce byly vybrány tři algoritmy: Multipop na zásobníku, fronta využívající dva zásobníky a Splay strom. Tyto algoritmy byly zvoleny tak, aby společně tvořily trojici příkladů různé obtížnosti a zároveň umožňovaly názorně ukázat různé projevy amortizované složitosti. Jejich základní charakteristiku shrnuje tabulka 2.6.1.

Tabulka 2.6.1: Vybrané algoritmy vhodné pro amortizovanou analýzu

| **Algoritmus** | **Obtížnost** | **Stručná charakteristika** |
|---|---|---|
| Multipop na zásobníku | jednoduchá | Jednodušší příklad vhodný pro úvod do amortizované analýzy, na němž lze ukázat, že i operace, která může být v jednom okamžiku dražší, zůstává v delší posloupnosti nákladově vyvážená. |
| Fronta využívající dva zásobníky | střední | Příklad implementace fronty pomocí dvou zásobníků, u něhož se náklady přirozeně rozkládají mezi operace enqueue a dequeue a kde se amortizovaná analýza uplatňuje při vysvětlení přesunů prvků mezi zásobníky. |
| Splay strom | vyšší | Pokročilejší datová struktura, u níž se amortizovaná analýza používá k popisu dlouhodobého chování po sérii přístupových operací a k vysvětlení vlivu rotací na budoucí náklady operací. |

Z uvedeného přehledu je patrné, že vybrané algoritmy pokrývají různé úrovně obtížnosti i různé způsoby projevu amortizované složitosti. Právě proto tvoří vhodný základ pro praktickou část této práce, v níž budou dále využity pro vytvoření názorné výukové vizualizace.

# Kapitola 3

# Návrh komponenty výukového serveru

Návrh výukové komponenty zaměřené na amortizovanou složitost vychází z potřeby propojit teoretický výklad s názornou a interaktivní formou prezentace algoritmů. Cílem takového návrhu je vytvořit prostředí, které bude studentům usnadňovat orientaci v dané problematice, podporovat aktivní práci s algoritmy a současně zohledňovat výukové, funkční i uživatelské požadavky, které jsou pro podobný typ aplikace podstatné.

## 3.1 Cíl a role navrhované komponenty

Navrhovaná komponenta představuje prostředek, jehož smyslem není pouze technické zobrazení vybraných algoritmů, ale především podpora jejich srozumitelného a názorného studia. Její význam spočívá v tom, že má studentům zprostředkovat problematiku amortizované složitosti způsobem, který propojí teoretické poznatky s konkrétním sledováním chování algoritmů a datových struktur.

### 3.1.1 Zařazení komponenty do výukového serveru

Vytvořená komponenta vzniká jako součást výukového serveru určeného pro předměty TI. Tento výukový server má mít podobu sady dynamických webových stránek, jejichž hlavním cílem je umožnit budoucím studentům lépe pochopit principy různých typů úloh a problémů prostřednictvím názorných a interaktivních ukázek. Na rozdíl od běžných statických výukových materiálů, které obvykle pracují pouze s pevně daným počtem příkladů, má takové řešení umožnit generovat libovolný počet ukázkových příkladů vybraných algoritmů na základě vstupů od uživatele a sledovat jejich průběh v různých situacích.

Navrhovaná komponenta je v rámci tohoto serveru zaměřena na oblast amortizované složitosti. Jejím úkolem je rozšířit výukový server o část, která studentům umožní lépe pochopit principy tohoto typu analýzy na konkrétních algoritmech a datových strukturách. Komponenta se tedy nesnaží představit samostatný izolovaný nástroj, ale stává se součástí výuky v TI a zapadá do širšího konceptu interaktivní podpory studia TI.

### 3.1.2 Hlavní účel komponenty

Hlavním účelem této navržené komponenty je vytvořit prostředek pro pochopení počítání složitosti vybraných algoritmů s využitím amortizované analýzy, která byla podrobně popsána v předchozí kapitole. Komponenta má budoucím studentům poskytnout vizualizaci toho, jak se algoritmus chová při zpracování vstupu, jak se mění stav dané datové struktury a jak probíhá vývoj hodnot v jednotlivých krocích, které jsou důležité pro amortizovanou analýzu. Důraz je kladen především na to, aby uživatel viděl podrobný průběh algoritmu v každém kroku, nikoli pouze konečný výsledek výpočtu.

Účelem komponenty tedy není pouze umožnit spuštění algoritmu, ale především podpořit hlubší pochopení celého principu, proč se amortizovaná složitost hodnotí v rámci delší posloupnosti operací. Komponenta má propojit teoretický výklad s názornou ukázkou chování algoritmu a nabídnout studentovi možnost sledovat souvislosti mezi jednotlivými operacemi a jejich dlouhodobým dopadem.

### 3.1.3 Přínos komponenty pro výuku amortizované složitosti

Celá problematika amortizované složitosti bývá pro studenty často obtížně pochopitelná, protože její správné pochopení vyžaduje sledování souvislostí mezi více operacemi, změn stavů vizualizovaných datových struktur a vývoje hodnot, které se při amortizované analýze používají. Klasické výklady v teoretické formě pomocí prezentací, skript nebo několika málo statických ukázek nemusí vždy stačit k tomu, aby si studenti vytvořili dostatečně jasnou představu o tom, proč se některé operace mohou jevit jako nákladné oproti ostatním a proč je přesto možné celkové chování algoritmu hodnotit příznivěji.

Důležitý přínos této komponenty spočívá především v propojení teoretického výkladu s názornou praktickou ukázkou. Student může při práci s komponentou aktivně sledovat průběh algoritmů, pozorovat změny stavů datových struktur a lépe chápat vztah mezi jednotlivými operacemi a jejich dlouhodobým chováním. Komponenta tak může přispět k názornější a srozumitelnější výuce amortizované složitosti v rámci předmětů TI.

## 3.2 Požadavky na komponentu

Při návrhu výukové komponenty bylo nutné zohlednit nejen to, co má výsledná aplikace technicky umožňovat, ale také jakým způsobem bude podporovat práci uživatele a jak účinně přispěje k pochopení sledované problematiky. Návrh proto musel vycházet z požadavků, které se týkaly funkčnosti, uživatelské srozumitelnosti i celkové výukové hodnoty navrhovaného řešení.

### 3.2.1 Funkční požadavky

Při návrhu celé komponenty bylo potřeba vycházet z požadavků, které byly stanoveny v zadání bakalářské práce. Mezi hlavní funkční požadavky patřila zejména podpora více algoritmů vhodných pro amortizovanou analýzu, možnost zadávat vstupy různými způsoby, zobrazení průběhu algoritmu po jednotlivých krocích, zobrazení aktuálního stavu datové struktury a současně i doplňujících hodnot důležitých pro interpretaci amortizované analýzy. Součástí návrhu bylo také to, aby u každého algoritmu bylo dostupné i statické odvození jeho složitosti jako doplnění interaktivní simulace. Hlavní funkční požadavky navrhované komponenty shrnuje tabulka 3.2.1.

Tabulka 3.2.1: Hlavní funkční požadavky navrhované komponenty

| **Požadavek** | **Stručný popis** |
|---|---|
| Podpora více algoritmů | Komponenta musí umožnit práci alespoň se třemi algoritmy, u nichž je vhodné analyzovat chování pomocí amortizované analýzy. |
| Různé způsoby zadávání vstupů | Uživatel musí mít možnost zadávat vstupy ručně, nechat je náhodně generovat, pracovat s ukázkovými vstupy různé velikosti a zadávat je také pomocí syntaxe příkazů. |
| Zobrazení průběhu algoritmu | Komponenta musí umožnit sledovat průběh výpočtu po jednotlivých krocích, nikoli pouze výsledný stav po dokončení algoritmu. |
| Zobrazení stavu datové struktury | V každém kroku simulace musí být zřetelně zobrazen aktuální stav datové struktury, s níž daný algoritmus pracuje. |
| Zobrazení doplňujících hodnot | Součástí simulace musí být zobrazení počtu kroků, přesného počtu provedených operací a hodnot souvisejících s amortizovanou analýzou, například potenciálu nebo naspořených mincí. |
| Podpora teoretického výkladu | Ke každému algoritmu musí být dostupné také odvození jeho složitosti, aby komponenta nesloužila jen k simulaci, ale i jako podpora teoretického pochopení. |

### 3.2.2 Požadavky na uživatelskou interakci, názornost a výukovou hodnotu

Vedle samotné funkčnosti bylo nutné navrhnout komponentu také tak, aby s ní dokázal pracovat každý uživatel přehledně a srozumitelně. Celý vzhled stránky měl působit přívětivě a zadávání vstupů mělo být navrženo tak, aby bylo přehledné a srozumitelné. Uživatel by neměl mít problém zadat libovolný vstup a v případě chyby při zadávání by měl být vhodným způsobem upozorněn na to, co zadává nesprávně.

Návrh zároveň počítal s tím, že uživatel bude moci měnit vstupy, opakovaně spouštět simulace a sledovat, jak se změny vstupu projeví v průběhu algoritmu. Důležitým požadavkem bylo také to, aby uživatel mohl snadno sledovat jednotlivé kroky výpočtu a rozuměl tomu, co se v daném okamžiku děje. Interakce s komponentou proto neměla být omezena pouze na zobrazení výsledku, ale měla podporovat průběžné vnímání změn stavu datové struktury a souvisejících hodnot.

Jedním z nejdůležitějších požadavků na navrhovanou komponentu byla současně její názornost a výuková hodnota. Nestačilo, aby komponenta technicky zobrazovala průběh algoritmu, ale bylo také nutné, aby uživateli pomáhala pochopit celou problematiku. Návrh proto musel směřovat k tomu, aby student mohl jasně sledovat vztah mezi jednotlivými operacemi, změnami stavu datové struktury, počtem provedených operací a vývojem hodnot důležitých pro použitou metodu analýzy.

Výuková hodnota komponenty spočívá především v propojení teoretického výkladu s praktickou ukázkou. Uživatel nemá pouze číst popis algoritmu, ale má současně vidět jeho konkrétní chování při různých typech vstupů a lépe chápat, proč se celkové náklady algoritmu neposuzují jen podle jedné operace. Proto bylo při návrhu důležité dbát na srozumitelnost, přehlednost a na takové uspořádání informací, které podpoří lepší orientaci uživatele v průběhu simulace.

## 3.3 Výběr algoritmů pro praktickou část

Volba algoritmů zařazených do praktické části měla zásadní význam pro celkové zaměření vytvářené komponenty. Bylo totiž potřeba zvolit takové příklady, které budou z hlediska amortizované analýzy dostatečně názorné, budou se lišit svou obtížností a současně umožní studentům postupně přecházet od jednodušších principů ke složitějším datovým strukturám.

### 3.3.1 Kritéria výběru algoritmů

Výběr algoritmů byl velmi důležitý, protože bylo potřeba zvolit takové příklady, které budou vhodné z hlediska amortizované analýzy a současně budou přínosné pro výuku. Jedním z hlavních kritérií bylo to, že u zvolených algoritmů nebo datových struktur se mohou jednotlivé operace výrazně lišit svou skutečnou cenou, avšak při delší posloupnosti operací lze jejich chování hodnotit příznivěji pomocí amortizované analýzy.

Dalším důležitým kritériem byla rozdílná obtížnost vybraných algoritmů. Cílem bylo zařadit takové příklady, které studentům umožní postupovat od jednoduššího principu k náročnějšímu. Z tohoto důvodu byl vybrán jeden jednodušší algoritmus, jeden středně obtížný příklad a jedna pokročilejší datová struktura. Výběr byl zároveň přizpůsoben tomu, aby bylo možné jednotlivé algoritmy vhodně vizualizovat a názorně na nich ukázat různé projevy amortizované složitosti.

### 3.3.2 Multipop na zásobníku jako jednoduchý příklad

Jako jednodušší příklad byl zvolen algoritmus Multipop na zásobníku. Tento příklad je pro výuku vhodný především proto, že na něm lze poměrně snadno ukázat základní princip amortizované analýzy. Zásobník je navíc datová struktura, která je studentům obvykle dobře známá, a proto není nutné věnovat velkou pozornost jejím základním vlastnostem. Díky tomu je možné soustředit se především na samotný princip analýzy nákladů jednotlivých operací.

V rámci tohoto příkladu se nad zásobníkem provádějí zejména operace *Push*, *Pop* a *Multipop*. Operace *Push* vloží prvek na vrchol zásobníku a operace *Pop* jeden prvek z vrcholu odebere. Obě tyto operace mají konstantní časovou složitost $O(1)$. Operace *Multipop* však může v jednom kroku odebrat více prvků najednou, a proto může mít v nejhorším případě lineární složitost vzhledem k počtu odebíraných prvků. Přesto lze ukázat, že v delší posloupnosti operací je amortizovaná cena jednotlivých operací konstantní, protože každý prvek může být ze zásobníku odstraněn pouze omezený početkrát. Právě tato vlastnost činí z Multipopu na zásobníku vhodný úvodní příklad pro pochopení amortizované analýzy [6].

### 3.3.3 Fronta využívající dva zásobníky jako středně obtížný příklad

Jako středně obtížný příklad byla zvolena fronta implementovaná pomocí dvou zásobníků. Tento příklad je vhodný především proto, že kombinuje známé principy datových struktur LIFO a FIFO a současně ukazuje, že i operace, která může být v konkrétním okamžiku nákladná, může mít při delší posloupnosti operací příznivou amortizovanou složitost.

Při této implementaci se pro realizaci fronty používají dva zásobníky. Do prvního zásobníku se prvky vkládají operací *Enqueue*, zatímco odebírání pomocí operace *Dequeue* probíhá ze zásobníku druhého. Pokud je druhý zásobník prázdný, musí se prvky z prvního zásobníku do druhého postupně přesunout. Operace *Enqueue* má konstantní složitost $O(1)$, zatímco operace *Dequeue* může mít v nejhorším případě lineární složitost $O(n)$, pokud je nutné převést větší počet prvků. Z dlouhodobého

hlediska však každý prvek projde tímto přesunem pouze jednou, a proto lze ukázat, že obě základní operace mají konstantní amortizovanou cenu. Tento algoritmus tak představuje vhodný střední stupeň mezi jednoduchým příkladem Multipopu a pokročilejší strukturou v podobě Splay stromu [7].

### 3.3.4 Splay strom jako pokročilejší příklad

Jako pokročilejší příklad byl zvolen Splay strom, tedy samočinně upravující se binární vyhledávací strom BST. Tento příklad představuje náročnější datovou strukturu, protože zde amortizovaná analýza neslouží pouze k vysvětlení jedné pomocné operace, ale k popisu dlouhodobého chování celé struktury při sérii přístupových operací.

Základní myšlenkou Splay stromu je, že po každém přístupu k uzlu dochází k jeho přesunu směrem ke kořeni pomocí rotací. Využívají se přitom operace typu *Zig*, *Zig-Zig* a *Zig-Zag*. Mezi základní operace nad touto strukturou patří *Search*, *Insert* a *Delete*. Operace *Search* slouží k vyhledání prvku ve stromu, operace *Insert* k vložení nového prvku a operace *Delete* k jeho odstranění. Pro Splay strom je typické, že po provedení těchto operací dochází k přesunu navštíveného nebo upravovaného uzlu směrem ke kořeni, čímž se mění struktura stromu s ohledem na budoucí přístupy.

Jedna konkrétní operace může být v nejhorším případě časově náročná a může dosáhnout až lineární složitosti $O(n)$, pokud se hledaný nebo upravovaný uzel nachází hluboko ve stromu. Přesto lze ukázat, že při delší posloupnosti operací mají základní operace nad Splay stromem amortizovanou složitost $O(\log n)$. Splay strom je proto vhodným příkladem pokročilejšího využití amortizované analýzy v oblasti dynamických datových struktur a tvoří přirozený vrchol trojice vybraných algoritmů [8].

## 3.4 Návrh funkcionality komponenty

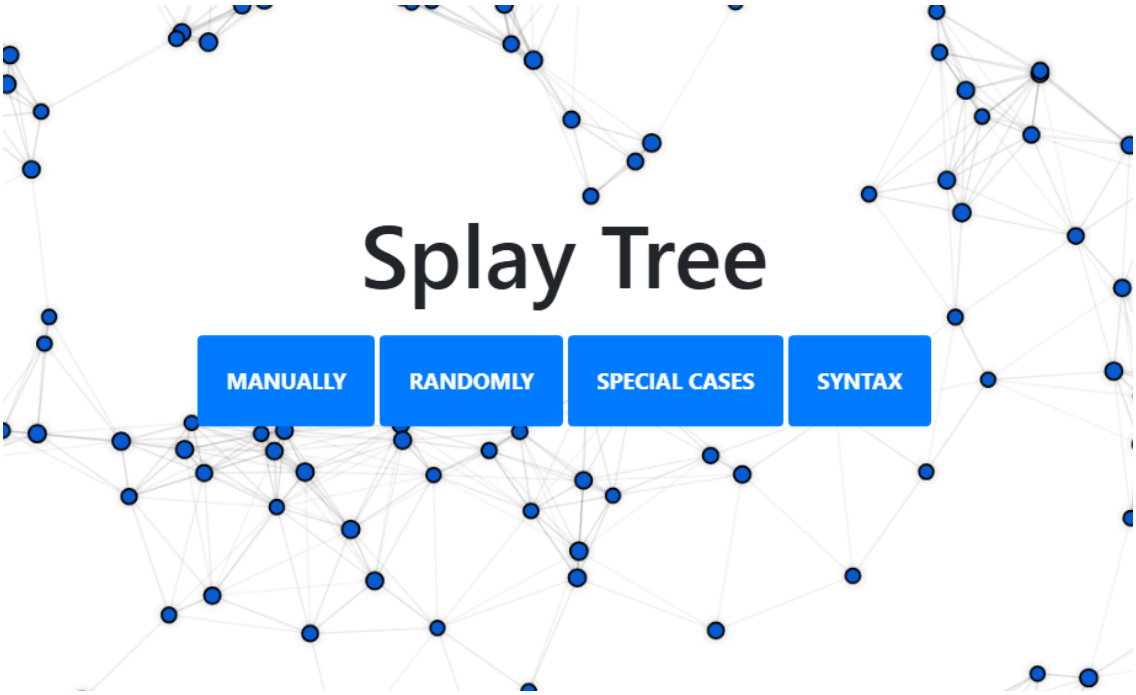
Funkcionalita navrhované komponenty představuje jednu z klíčových částí celého řešení, protože určuje, jakým způsobem bude uživatel s aplikací pracovat a jak efektivně mu bude umožněno sledovat chování vybraných algoritmů. Při návrhu bylo proto nutné zohlednit nejen samotné spuštění algoritmů, ale také formu práce se vstupy, způsob prezentace jednotlivých kroků a rozsah informací, které mají uživateli pomoci při pochopení amortizované analýzy.

### 3.4.1 Režimy zadávání vstupů

Velmi důležitou součástí návrhu funkcionality komponenty bylo umožnit uživateli pracovat s více způsoby zadávání vstupů. Pro vyzkoušení vybraných algoritmů by byl pouze jeden způsob zadávání vstupů nedostačující, a proto bylo vhodné navrhnout více typů vstupu, které budou odpovídat různým potřebám uživatele i různým výukovým situacím. Komponenta proto byla navržena tak, aby umožňovala ruční zadávání vstupů, generování náhodných vstupů podle zvolených parametrů, práci s ukázkovými vstupy reprezentujícími typické situace a také zadávání vstupů pomocí syntaxe příkazů.

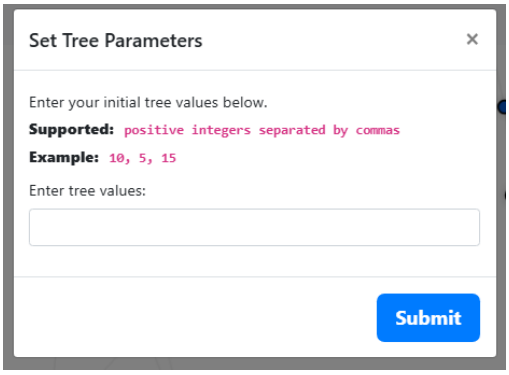
Tyto navržené typy zadávání vstupů umožňují studentovi pracovat s komponentou aktivnějším způsobem. Uživatel si může sám zvolit, zda chce sledovat předem připravený příklad, experimentovat s náhodně vytvořeným vstupem, nebo si přesně nadefinovat vlastní scénář pomocí ručně zadaných hodnot nebo posloupnosti příkazů. Návrh této části funkcionality tak podporuje jak samostatné objevování chování algoritmů, tak cílené sledování konkrétních situací důležitých pro pochopení amortizované složitosti.

Na obrázku 3.4.1 je znázorněna hlavní nabídka režimů zadávání vstupů, která je uživateli k dispozici po zvolení konkrétního algoritmu.

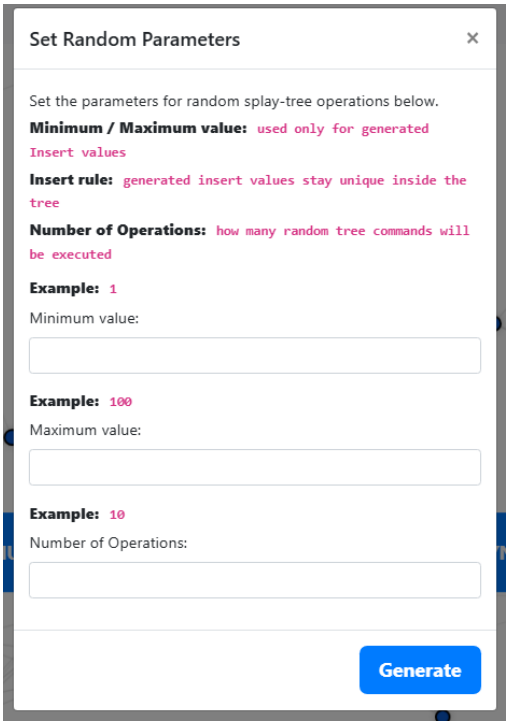



Obrázek 3.4.1: Hlavní nabídka režimů zadávání vstupů u zvoleného algoritmu

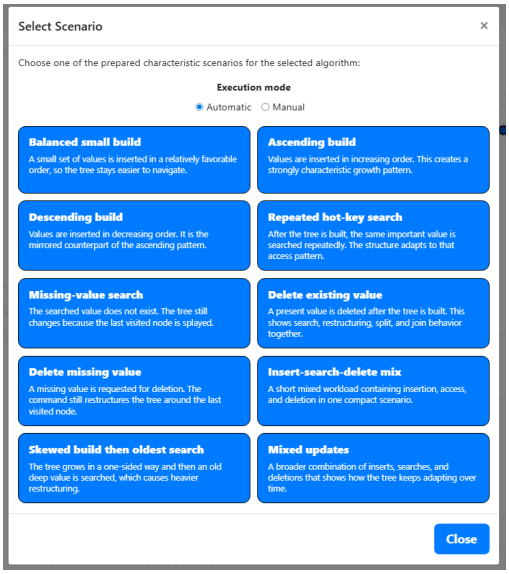
Obrázek 3.4.2 pak ukazuje konkrétní podobu jednotlivých režimů zadávání, konkrétně ruční zadávání vstupů, náhodné generování, volbu typického případu a zadávání vstupů pomocí syntaxe příkazů.



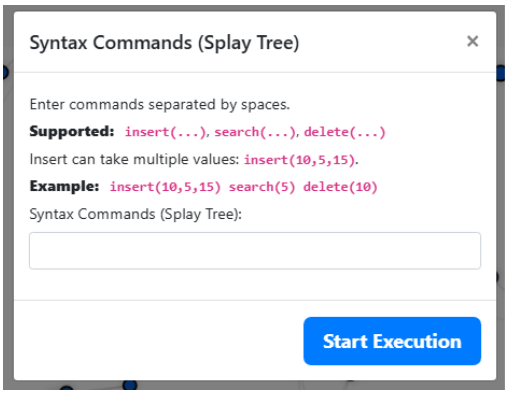

(a) Ruční zadávání vstupů




(b) Náhodné generování vstupů




(c) Volba typického případu




(d) Zadávání vstupů pomocí syntaxe příkazů

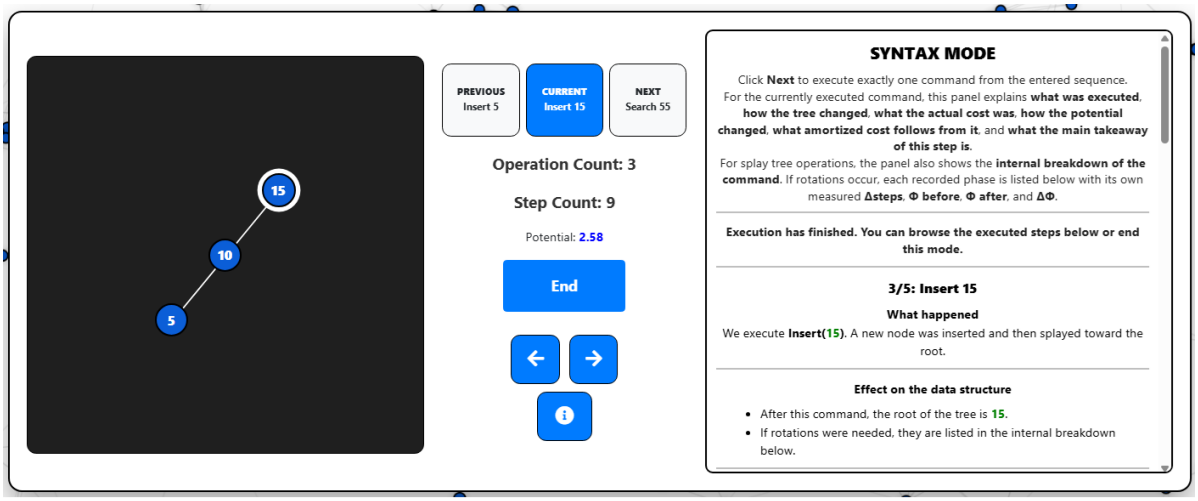
Obrázek 3.4.2: Příklady jednotlivých režimů zadávání vstupů

### 3.4.2 Zobrazení průběhu algoritmu po jednotlivých krocích

Další důležitou součástí návrhu funkcionality komponenty bylo zobrazení průběhu algoritmu po jednotlivých krocích. Pro výuku amortizované složitosti totiž nestačí uživateli ukázat pouze konečný výsledek, ale je nutné umožnit mu sledovat, co se děje v průběhu celého výpočtu. Komponenta proto byla navržena tak, aby uživatel mohl vidět postupný vývoj algoritmu a změny stavu datové struktury v jednotlivých krocích simulace.

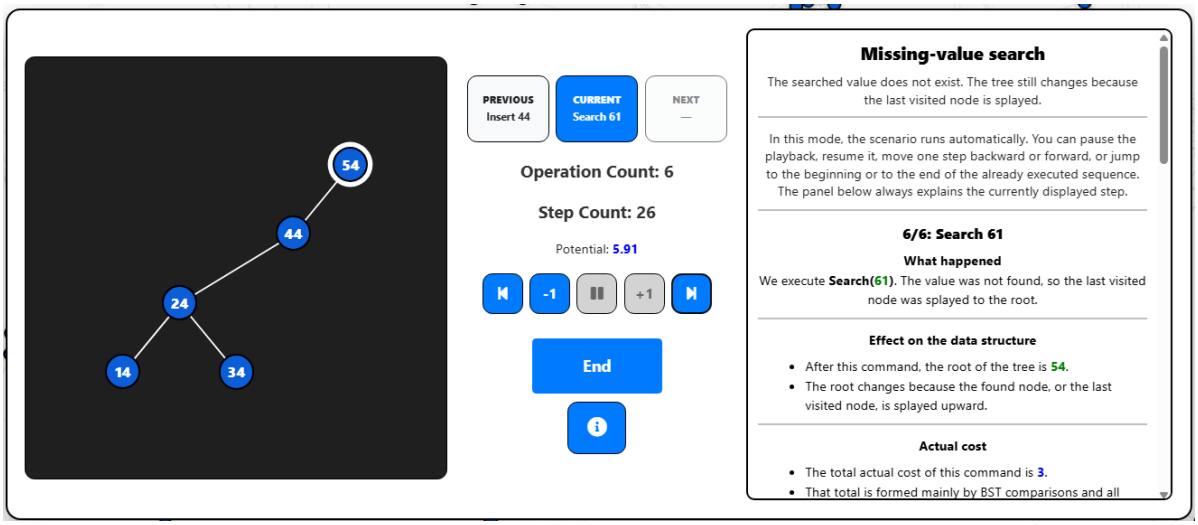
Takové zobrazení je důležité především proto, že studentovi umožňuje lépe pochopit návaznost jednotlivých operací a jejich vliv na další chování algoritmu. Díky krokovému průběhu lze sledovat nejen samotný výsledek operace, ale i způsob, jakým k němu algoritmus dospívá. Návrh této funkcionality proto představuje jednu z klíčových částí celé komponenty.

Na obrázku 3.4.3 je znázorněn průběh algoritmu v režimu syntaxe, kde uživatel postupuje jednotlivými kroky simulace a může sledovat aktuální stav datové struktury i informace o právě prováděné operaci.




Obrázek 3.4.3: Zobrazení průběhu algoritmu po jednotlivých krocích v režimu syntaxe

Obrázek 3.4.4 pak ukazuje automatický průběh v režimu vybraného typického případu, kde se předem připravený scénář vykonává postupně bez nutnosti ručního zadávání každého kroku.



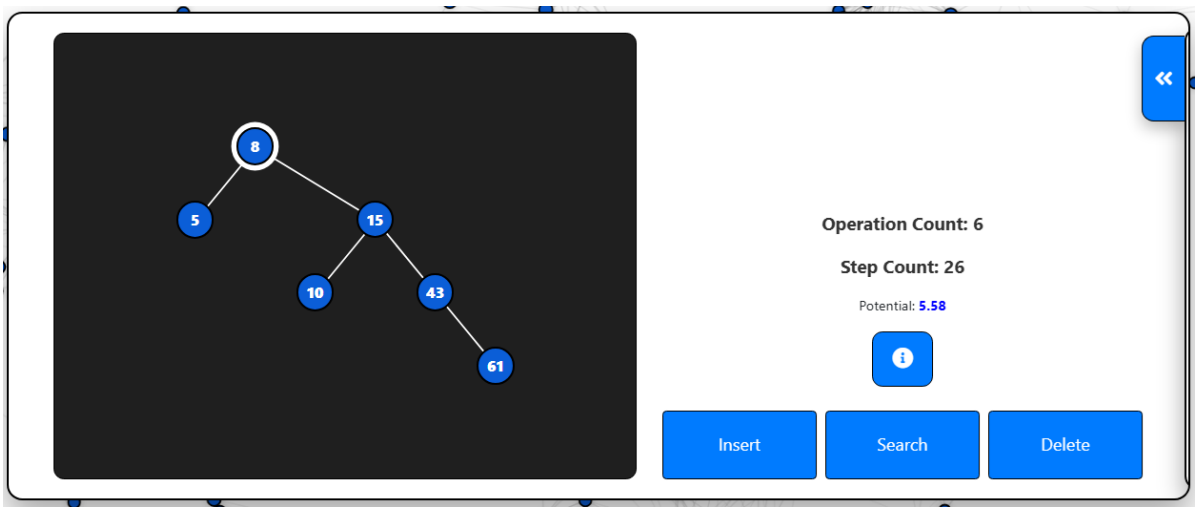

Obrázek 3.4.4: Automatický průběh simulace v režimu vybraného typického případu

### 3.4.3 Zobrazení počtu kroků, počtu operací a potenciálu

Pro správné pochopení amortizované analýzy bylo při návrhu důležité zohlednit také zobrazení hodnot, které s tímto typem hodnocení přímo souvisejí. Komponenta proto měla zobrazovat počet provedených kroků, přesný počet provedených operací a zároveň i hodnotu potenciálu, případně jinou odpovídající veličinu podle použité metody analýzy. Tyto hodnoty nemají sloužit pouze jako doplněk vizualizace, ale jako důležitá součást výkladu toho, proč se chování algoritmu hodnotí určitým způsobem.

Zobrazení počtu kroků, počtu operací a potenciálu umožňuje uživateli sledovat nejen to, co algoritmus právě vykonává, ale také to, jak se mění hodnoty důležité pro celkovou interpretaci jeho amortizované složitosti. Student tak může lépe pochopit vztah mezi jednotlivými operacemi, jejich skutečnými náklady a dlouhodobým chováním algoritmu v rámci celé posloupnosti operací.

Na obrázku 3.4.5 je ukázáno zobrazení počtu provedených kroků, přesného počtu provedených operací a aktuální hodnoty potenciálu v průběhu simulace. Tyto informace jsou součástí hlavního rozhraní komponenty a tvoří důležitou oporu při sledování vývoje algoritmu i při interpretaci jeho amortizované složitosti.



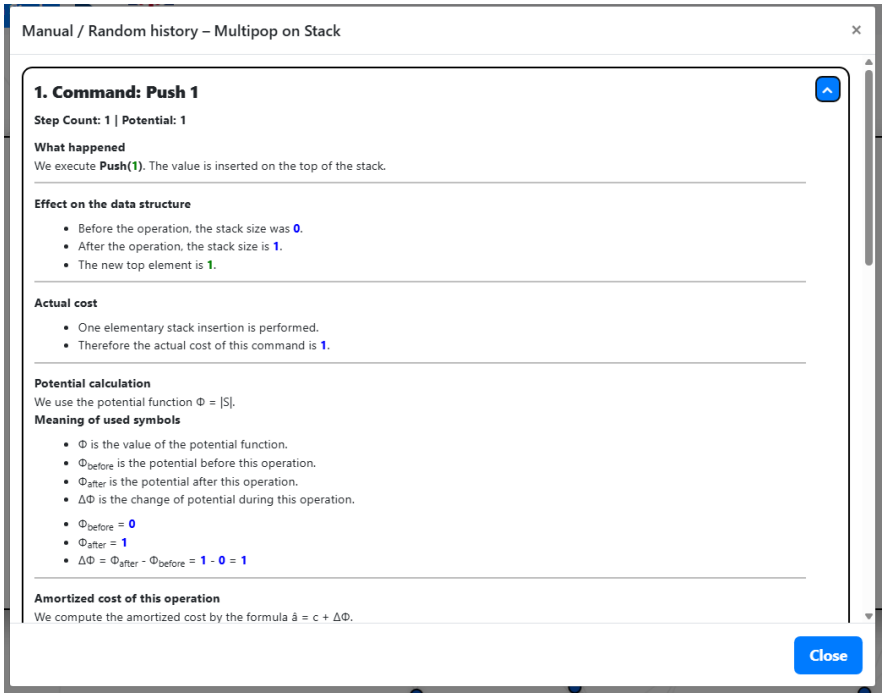

Obrázek 3.4.5: Zobrazení počtu kroků, počtu operací a aktuální hodnoty potenciálu v průběhu simulace

### 3.4.4 Zobrazení doplňujících informací o aktuálním stavu algoritmu

Součástí návrhu funkcionality komponenty bylo také zobrazení doplňujících informací o aktuálním stavu algoritmu. Samotná vizualizace datové struktury nebo zobrazení provedené operace totiž nemusí být vždy dostačující k tomu, aby uživatel přesně porozuměl tomu, co se v daném okamžiku děje. Z tohoto důvodu bylo vhodné doplnit průběh simulace o textové a přehledně uspořádané informace, které uživateli usnadní orientaci.

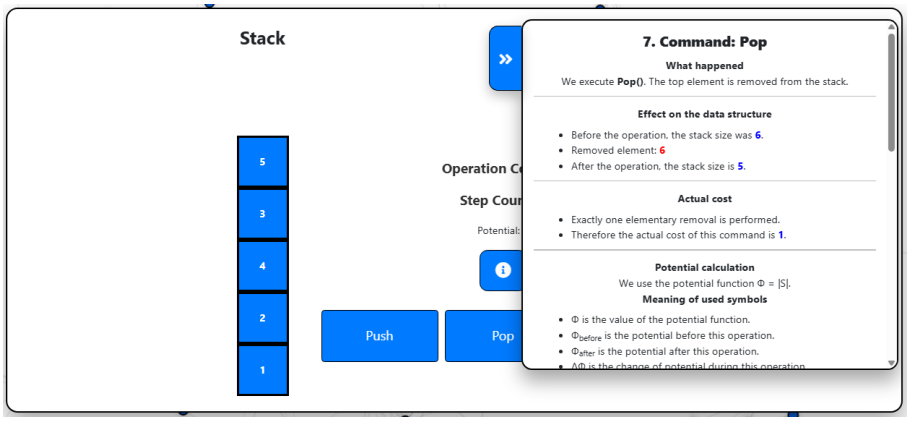
Tyto doplňující informace mají pomoci objasnit význam aktuálního kroku, stav datové struktury nebo souvislosti mezi právě prováděnou operací a chováním algoritmu jako celku. Uživatel tak nemá k dispozici pouze vizuální podobu aktuálního stavu, ale také slovní vysvětlení toho, co bylo provedeno, co se změnilo a jaký dopad má daný krok z hlediska amortizované analýzy. Návrh této části funkcionality tak podporuje srozumitelnost komponenty a přispívá k tomu, aby uživatel mohl lépe propojit vizuální průběh simulace s teoretickým výkladem algoritmu.

Na obrázku 3.4.6 je znázorněn příklad detailních informací dostupných v historii vykonaných kroků.




Obrázek 3.4.6: Detailní doplňující informace dostupné v historii provedených kroků

Obrázek 3.4.7 pak ukazuje doplňující informační panel přímo v průběhu práce s algoritmem, kde jsou zobrazeny informace vztahující se k aktuálně prováděné operaci.




Obrázek 3.4.7: Doplňující informační panel zobrazující aktuální stav operace při práci s algoritmem

## 3.5 Návrh uživatelského rozhraní

Uživatelské rozhraní navrhované komponenty představuje důležitou část celého řešení, protože významně ovlivňuje, jak snadno se bude uživatel v aplikaci orientovat a jak přirozeně bude moci pracovat s jednotlivými algoritmy. Při jeho návrhu proto bylo nutné zohlednit nejen přehlednost a srozumitelnost ovládání, ale také to, aby vizuální podoba celé komponenty podporovala její výukový charakter a usnadňovala pochopení sledované problematiky.

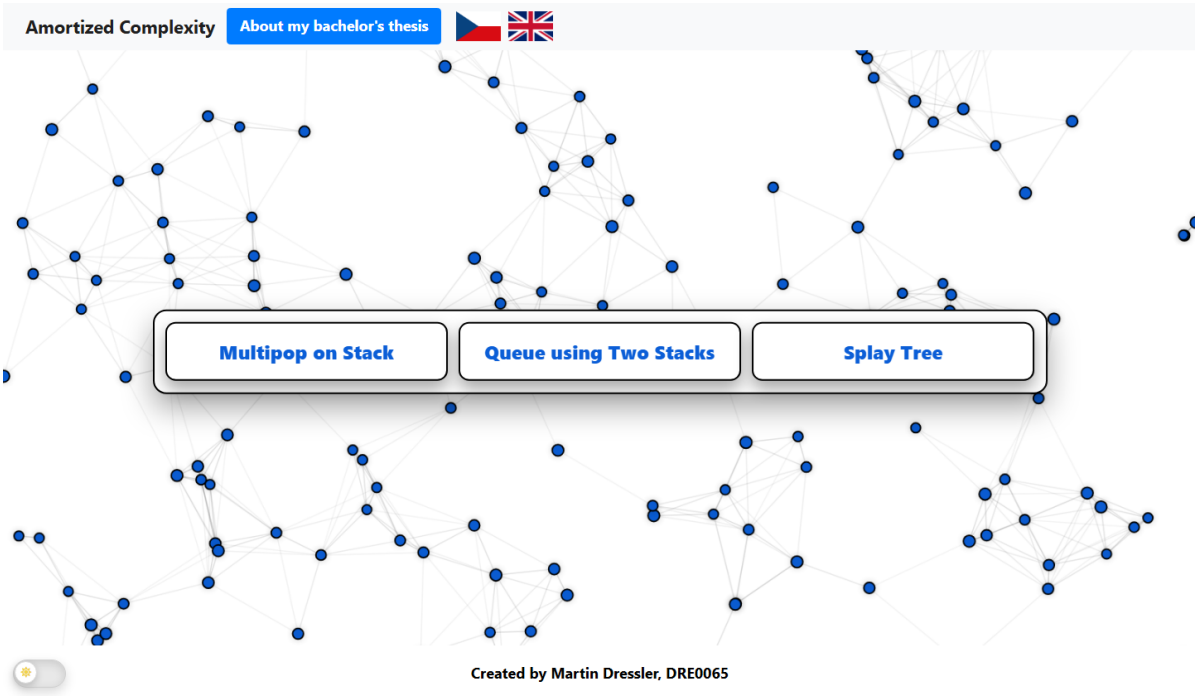
### 3.5.1 Návrh hlavní stránky a výběru algoritmu

Při návrhu uživatelského rozhraní bylo důležité vytvořit hlavní stránku, která bude pro uživatele přehledná a umožní mu rychlou orientaci v celé komponentě. Hlavní stránka proto byla navržena jako výchozí rozcestník, z něhož si uživatel vybírá konkrétní algoritmus, se kterým chce dále pracovat. Současně bylo cílem, aby již na této úrovni bylo zřejmé, že komponenta slouží jako součást výukového prostředí zaměřeného na amortizovanou složitost.

Velký důraz byl kladen na jednoduchost výběru jednotlivých algoritmů. Uživatel má mít možnost z hlavní stránky přímo přejít k vybranému algoritmu, aniž by musel procházet složitější strukturu nabídek. Návrh proto počítal s výrazným zobrazením všech tří algoritmů zařazených do praktické části práce. Tím je zajištěno, že uživatel ihned vidí, jaké algoritmy jsou v komponentě dostupné, a může si zvolit ten, který chce právě vyzkoušet.

Součástí návrhu hlavní stránky byly také další prvky podporující orientaci a dostupnost informací. Patří mezi ně zejména možnost přepínání jazykových preferencí a přístup k doplňujícím informacím o samotné bakalářské práci. Hlavní stránka tak neslouží pouze jako technický vstup do aplikace, ale zároveň plní orientační a informační funkci.

Na obrázku 3.5.1 je znázorněna hlavní stránka komponenty, na níž jsou zobrazeny dostupné algoritmy, přepínač jazykových preferencí a tlačítko pro zobrazení informací o bakalářské práci.



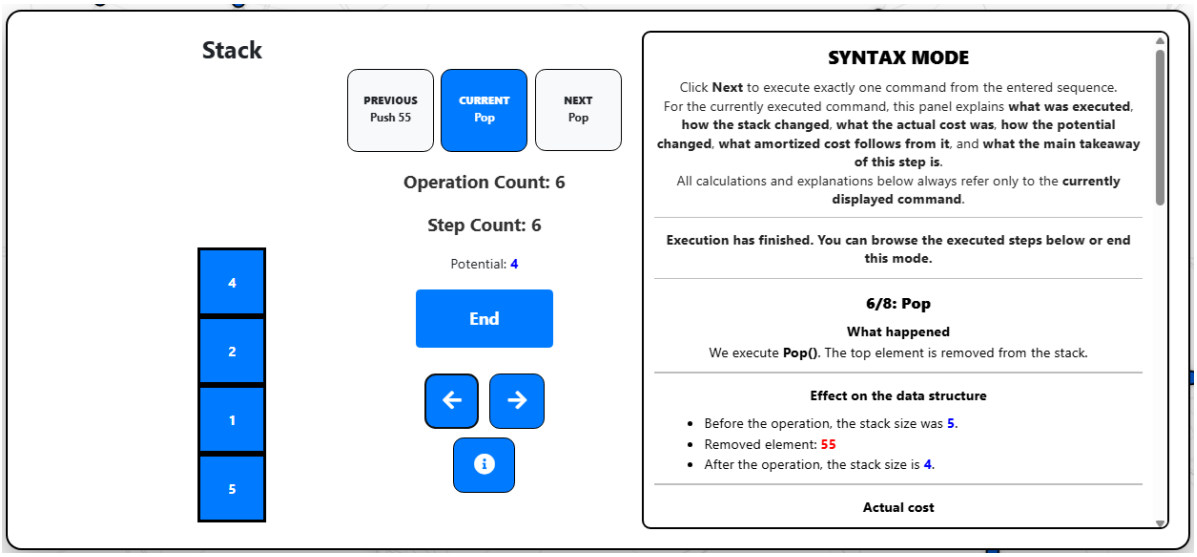

Obrázek 3.5.1: Hlavní stránka komponenty s výběrem algoritmů, přepínačem jazykových mutací a tlačítkem pro informace o bakalářské práci

### 3.5.2 Návrh ovládacích prvků simulace

Součástí návrhu uživatelského rozhraní bylo také vytvoření vhodných ovládacích prvků simulace. Tyto prvky musely uživateli umožnit pohodlně řídit průběh algoritmu a současně podporovat srozumitelné sledování jednotlivých kroků. Návrh proto počítal s tím, že uživatel nebude pouze pasivně sledovat vykonávání algoritmu, ale bude moci průběh simulace aktivně ovlivňovat.

Důležitou součástí ovládání bylo zejména krokování mezi jednotlivými stavy simulace. Uživatel má mít možnost přecházet na předchozí nebo následující krok, ukončit aktuální režim a podle potřeby si zobrazit také doplňující historii informací. Ovládací prvky proto byly navrženy tak, aby byly snadno dostupné a přehledně oddělené od samotné vizualizace datové struktury. Tím je zajištěno, že uživatel může současně sledovat stav algoritmu i pohodlně pracovat s jeho ovládáním.

Na obrázku 3.5.2 je znázorněn příklad návrhu ovládacích prvků simulace v režimu syntaxe. Součástí rozhraní jsou tlačítka pro pohyb mezi jednotlivými kroky, tlačítko pro ukončení režimu a tlačítko pro zobrazení historie informací.




Obrázek 3.5.2: Příklad ovládacích prvků simulace v režimu syntaxe

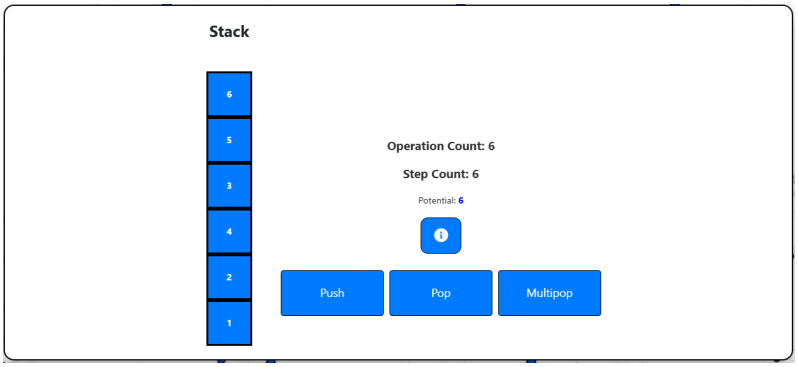
### 3.5.3 Návrh vizualizace datových struktur

Důležitou součástí návrhu uživatelského rozhraní bylo také zobrazení datových struktur používaných jednotlivými algoritmy. Cílem nebylo pouze zobrazit stav výpočtu formou číselných hodnot, ale navrhnout takovou vizualizaci, která uživateli umožní co nejlépe pochopit změny struktury v průběhu simulace. Z tohoto důvodu byla podoba vizualizace přizpůsobena konkrétnímu typu datové struktury, s níž daný algoritmus pracuje.

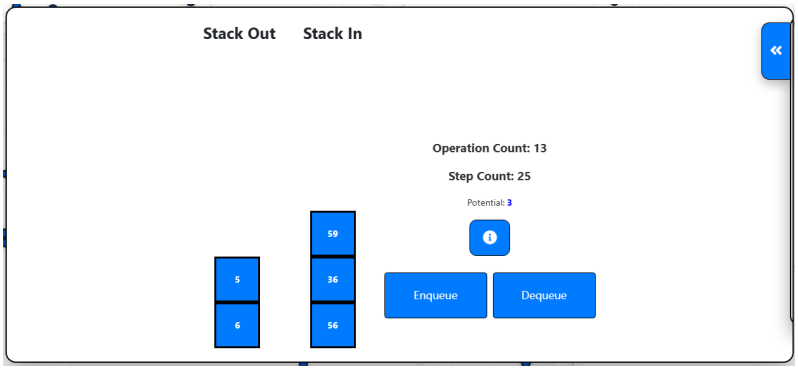
U algoritmu Multipop na zásobníku byla zvolena jednoduchá svislá reprezentace zásobníku, která přehledně ukazuje pořadí prvků i změny na vrcholu zásobníku. U fronty využívající dva zásobníky bylo naopak důležité oddělit oba zásobníky tak, aby bylo zřejmé, do které části se prvky vkládají a ze které se odebírají. U Splay stromu pak bylo nezbytné navrhnout stromovou vizualizaci, která umožní sledovat nejen obsah uzlů, ale i jejich vzájemné propojení a změny struktury při jednotlivých operacích.

Takto navržená vizualizace datových struktur podporuje názornost celé komponenty a umožňuje uživateli lépe porozumět tomu, jak se algoritmus chová při různých operacích. Uživatel tak nesleduje pouze výsledky výpočtu, ale i konkrétní změny uspořádání dat, které jsou pro pochopení amortizované složitosti důležité.

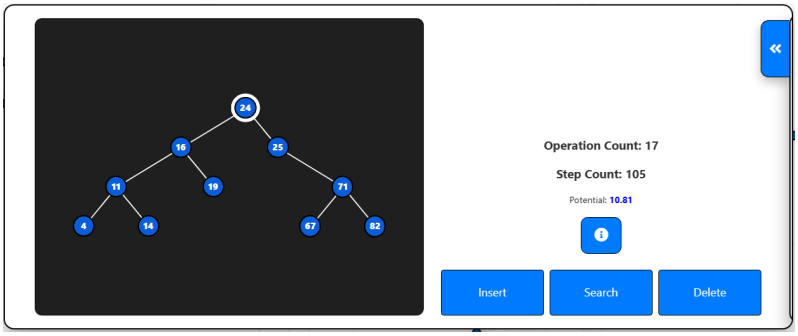
Na obrázku 3.5.3 jsou znázorněny příklady vizualizace datových struktur použitých v jednotlivých algoritmech, konkrétně zásobníku, fronty realizované pomocí dvou zásobníků a Splay stromu.




(a) Zásobník




(b) Fronta využívající dva zásobníky




(c) Splay strom

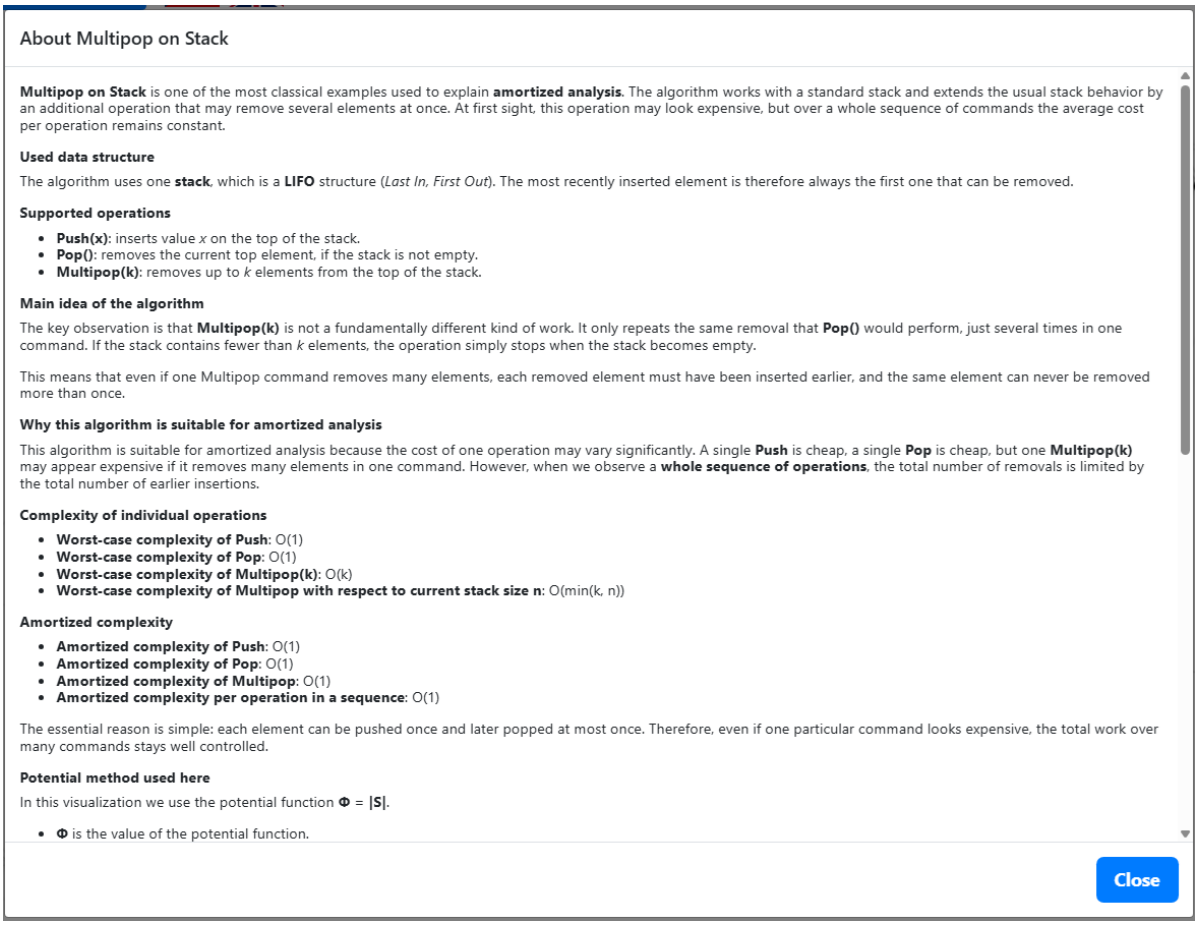
Obrázek 3.5.3: Příklady vizualizace datových struktur použitých v navrhované komponentě

### 3.5.4 Návrh informačních panelů a doprovodných vysvětlení o algoritmech

Součástí návrhu uživatelského rozhraní byly také informační panely a doprovodná vysvětlení vztahující se k jednotlivým algoritmům. Cílem těchto částí rozhraní nebylo pouze doplnit simulaci o další text, ale nabídnout uživateli přehledné vysvětlení základních vlastností algoritmu, použitých operací a jeho vztahu k amortizované analýze. Díky tomu může uživatel získat potřebný teoretický kontext ještě před samotnou prací se simulací nebo v jejím průběhu.

Návrh těchto informačních panelů vycházel z potřeby propojit praktickou ukázku s doprovodným výkladem. Uživatel tak nemá k dispozici pouze vizualizaci datové struktury a ovládací prvky simulace, ale také souvislý popis algoritmu, jeho hlavní myšlenky a vysvětlení, proč je daný algoritmus vhodný pro amortizovanou analýzu. Takové řešení podporuje lepší orientaci v problematice a současně posiluje výukový charakter celé komponenty.

Na obrázku 3.5.4 je znázorněn příklad informačního panelu věnovaného jednomu z vybraných algoritmů. Panel obsahuje základní popis algoritmu, přehled jeho operací, hlavní princip fungování i stručné vysvětlení jeho složitosti z hlediska amortizované analýzy.




Obrázek 3.5.4: Příklad informačního panelu s doprovodným vysvětlením vybraného algoritmu

## 3.6 UML komponentový diagram aplikace

Pro lepší přehled o navržené struktuře celé aplikace byl uveden také UML komponentový diagram, který zachycuje hlavní části implementované komponenty a vztahy mezi nimi. Hlavním účelem tohoto diagramu je především to, aby bylo jasné, jak jsou jednotlivé soubory a logické celky aplikace rozděleny, jaké jsou jejich role a jak mezi sebou navzájem spolupracují.

Aplikace je založena na modulárním rozdělení, kdy jsou společné funkce odděleny od částí specifických pro jednotlivé algoritmy. Toto uspořádání zjednodušuje orientaci v implementaci, podporuje přehlednost kódu a zároveň usnadňuje další rozšiřování nebo úpravy aplikace. Diagram se nesnaží pouze vizualizovat rozdělení souborů, ale především celkovou architekturu navrženého řešení.

Hlavní komponentou v diagramu je *Main Page*, která odpovídá hlavní stránce celé aplikace, tedy souboru `index.html`. Tato komponenta představuje základní bod celé aplikace, protože definuje hlavní rozložení stránky, načítá sdílené prostředky, aplikační logiku i části související s obsluhou parametrů a režimů spuštění. Z pohledu UI tvoří hlavní kostru aplikace, na niž navazují všechny další části.

Další částí jsou *Shared Resources*, tedy sdílené prostředky, které jsou využívány napříč celou aplikací. Do této komponenty patří především soubory `styles.css`, `animation.js` a `translations.js`. Tyto soubory poskytují vzhled aplikace, animační efekty v pozadí a jazykovou podporu. Jedná se tedy o vrstvu, která poskytuje podporu celému systému.

Velice důležitou částí je *Application*, která reprezentuje hlavní aplikační logiku. Zahrnuje soubory jako `state_app.js`, `language_app.js`, `modals_app.js`, `navigation_app.js`, `theme_app.js` a `init_app.js`. Tato část zajišťuje správu globálního stavu aplikace, přepínání jazyků, práci s modálními okny, navigaci mezi jednotlivými částmi a celkovou inicializaci aplikace po načtení stránky. V diagramu je zobrazena jako centrální komponenta, u které by se dalo říci, že tvoří střed celé aplikace, protože pracuje s většinou částí celého diagramu.

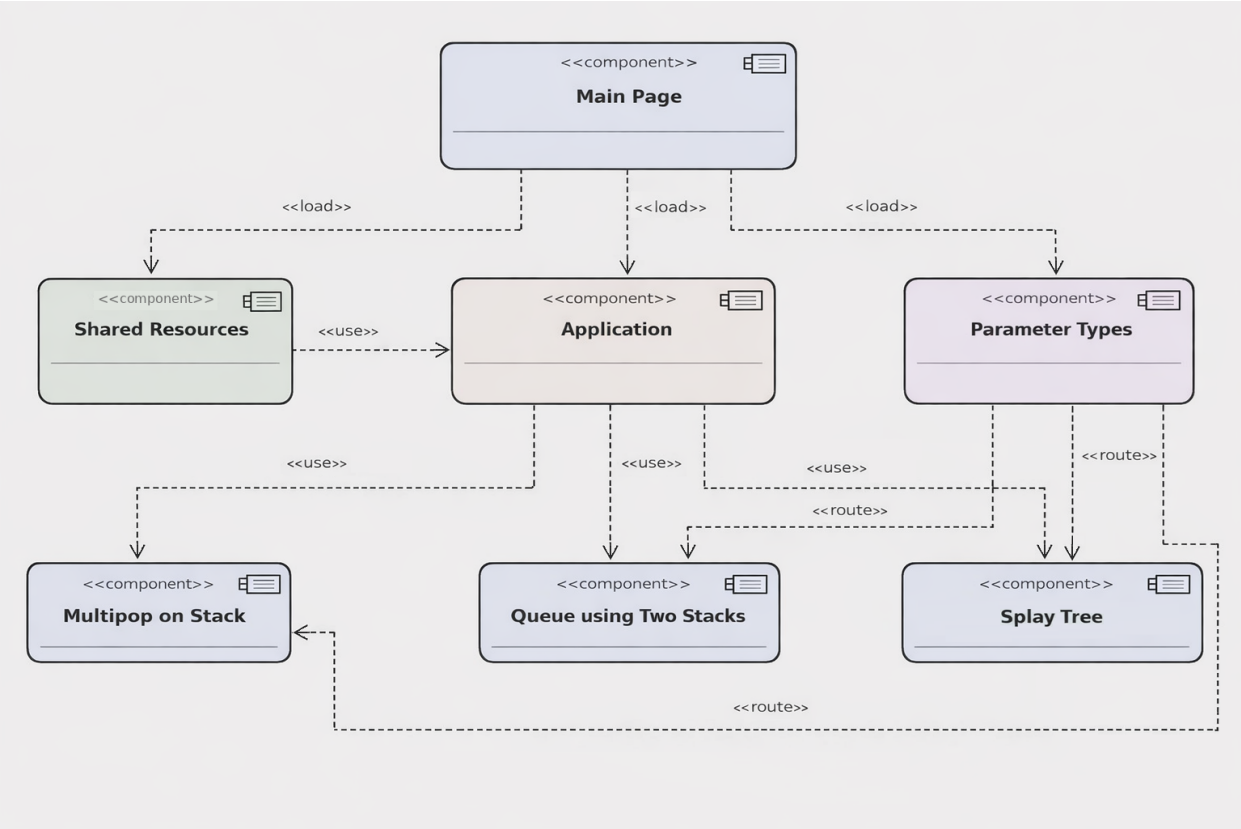
Další částí je *Parameter Types*. Tato část odpovídá souborům z adresáře `typesParams`. Patří sem například soubory `helpers_typesParams.js`, `manual_typesParams.js`, `random_typesParams.js`, `specialCases_typesParams.js` a `syntax_typesParams.js`. Tato část slouží ke zpracování různých typů vstupů a režimů spuštění aplikace.

Ve spodní části diagramu jsou umístěny tři samostatné algoritmy: *Multipop on Stack*, *Queue using Two Stacks* a *Splay Tree*. Každý z těchto algoritmů reprezentuje svou vlastní vizualizovanou úlohu a obsahuje vlastní stavovou logiku, vykreslování, obsluhu operací i podporu jednotlivých režimů spuštění. Do každé z těchto tří částí spadají soubory typu `state_...`, `ui_...`, `manual_...`, `actions_...`, `syntax_...`, `random_...` a `specialCases_...`. I když má každý z nich odlišné chování i vizualizaci, jejich vnitřní členění je navrženo stejným způsobem.

Zobrazené vazby v diagramu vyjadřují základní vztahy mezi komponentami. Vazby typu *load* znázorňují načítání hlavních částí aplikace ze vstupní stránky. Vazby typu *use* vyjadřují využívání sdílených nebo aplikačních služeb jednotlivými moduly. Vazby typu *route* poté reprezentují směrování zvoleného typu vstupu nebo režimu práce ke konkrétním algoritmům.

Toto rozdělení bylo zvoleno záměrně, aby bylo možné přehledně oddělit implementaci jednotlivých algoritmů od společných částí. Hlavní výhodou je poté snadnější správa kódu a také lepší orientace v celé implementaci při budoucích úpravách.

Na obrázku 3.6.1 je zachycena výsledná architektura aplikace v podobě UML komponentového diagramu. Diagram shrnuje hlavní části implementace a jejich vazby, přičemž názorně ukazuje oddělení sdílených prostředků, centrální aplikační logiky, zpracování vstupních režimů a samostatných algoritmů.




Obrázek 3.6.1: UML komponentový diagram aplikace

## 3.7 Návrh struktury komponenty

Celá struktura komponenty byla navržena tak, aby bylo možné oddělit prvky společné pro všechny algoritmy od částí, které jsou specifické vždy pro konkrétní algoritmus. Mezi společné prvky patří zejména hlavní stránka, výběr algoritmů, základní rozložení UI, obecné ovládací prvky a některé informační panely. Naopak část specifická pro daný algoritmus zahrnuje samotnou logiku jednotlivých algoritmů, jejich konkrétní operace, způsob vizualizace příslušné datové struktury a doprovodné informace vztahující se k danému algoritmu. Díky tomuto rozdělení je možné zachovat jednotný vzhled a princip ovládání celé komponenty, ale zároveň ponechat dostatečný prostor pro přizpůsobení jednotlivým algoritmům.

Při návrhu byl brán zřetel také na návaznost jednotlivých částí z pohledu uživatele. Práce s komponentou začíná na hlavní stránce, kde si uživatel zvolí konkrétní algoritmus. Poté si vybere typ vstupu a sleduje průběh algoritmu, hodnoty důležité pro amortizovanou analýzu a doplňující vysvětlení. Jednotlivé části komponenty byly vytvořeny tak, aby na sebe logicky navazovaly a vytvářely ucelené prostředí pro práci uživatele.

Při návrhu struktury bylo vhodné zohlednit také možnost budoucího rozšíření. Pokud by bylo potřeba aplikaci v budoucnu dále rozšířit, neměl by být problém přidat další algoritmy vhodné pro amortizovanou analýzu, aniž by bylo nutné zásadně měnit základní princip fungování celé komponenty. Stejným způsobem by bylo možné rozšiřovat informační panely, přidávat nové režimy práce se vstupy a další prvky.

# Kapitola 4

# Implementace komponenty

Navržená komponenta výukového serveru pro oblast TI byla realizována jako dynamická webová aplikace, která klade důraz na přehlednost, modularitu a interaktivní práci uživatele. Implementace vychází z rozdělení aplikace na samostatné logické celky, které zajišťují jak aplikační logiku, práci s UI, tak i chování jednotlivých algoritmů. Důležitým aspektem tohoto řešení je propojení vizualizace s reálným průběhem výpočtu, kdy jsou jednotlivé kroky algoritmů přímo zobrazovány a aktualizovány prostřednictvím manipulace s DOM. Celá aplikace byla navržena tak, aby byla snadno rozšiřitelná, přehledná z pohledu implementace a zároveň vhodná pro praktické využití ve výuce.

## 4.1 Použité technologie a vývojové prostředí

Implementace komponenty byla realizována jako dynamická webová aplikace s využitím technologií HTML, CSS a JS. Jazyk HTML slouží pro definici struktury celé stránky, CSS pro její vizuální styl a responzivní zobrazení a JS pro zajištění aplikační logiky. Dynamická práce s obsahem stránky je realizována prostřednictvím manipulace s DOM, což umožňuje aktualizaci jednotlivých částí UI bez nutnosti opětovného načítání celé stránky. Pro vizuální animační efekty v pozadí stránky bylo využito také Canvas API.

V implementaci byly také využity vybrané knihovny, které usnadnily práci s UI. Pro stylování a rozložení stránky byla použita knihovna Bootstrap, pro práci s ikonami knihovna Font Awesome a pro podporu JS komponent Bootstrapu knihovna jQuery spolu s knihovnou Popper.js. Aplikace nevyužívá žádné externí API, veškerá logika je realizována na straně klienta.

Celá praktická část byla vyvíjena v prostředí Visual Studio Code, které sloužilo pro implementaci aplikace. Teoretická část práce byla zpracována pomocí systému LaTeX v prostředí Overleaf.

## 4.2 Struktura a realizace aplikace

Aplikace je rozdělena do více zdrojových souborů podle jejich účelu. Toto rozdělení zlepšuje přehlednost implementace, odděluje společné části od specializovaných modulů a usnadňuje další úpravy i rozšiřování celé aplikace.

### 4.2.1 Organizace zdrojových souborů

Zdrojové soubory jsou rozděleny na hlavní stránku, sdílené prostředky, aplikační logiku, zpracování vstupních režimů a moduly jednotlivých algoritmů. Díky tomu lze snadno rozlišit, která část zajišťuje konkrétní chování aplikace.

- `index.html` – hlavní struktura stránky
- `styles.css` – vzhled a responzivita
- `animation.js` – animované pozadí stránky
- `translations.js` – jazykové překlady aplikace
- `state_app.js` – globální stav aplikace
- `language_app.js` – přepínání jazyků
- `modals_app.js` – obsluha modálních oken
- `navigation_app.js` – navigace mezi částmi
- `theme_app.js` – vzhledové nastavení
- `init_app.js` – inicializace aplikace
- `helpers_typesParams.js` – pomocné funkce vstupů
- `manual_typesParams.js` – ruční zadávání vstupu
- `random_typesParams.js` – náhodné generování vstupu
- `specialCases_typesParams.js` – ukázkové speciální případy
- `syntax_typesParams.js` – vstup pomocí příkazů
- `state_multipop.js` – globální stav algoritmu Multipop na zásobníku
- `ui_multipop.js` – rozhraní algoritmu Multipop na zásobníku
- `manual_multipop.js` – ruční režim algoritmu Multipop na zásobníku

- `actions_multipop.js` – operace algoritmu Multipop na zásobníku
- `syntax_multipop.js` – syntaxe algoritmu Multipop na zásobníku
- `random_multipop.js` – náhodný režim algoritmu Multipop na zásobníku
- `specialCases_multipop.js` – speciální případy algoritmu Multipop na zásobníku
- `state_queue2Stacks.js` – globální stav algoritmu Fronta pomocí dvou zásobníků
- `ui_queue2Stacks.js` – rozhraní algoritmu Fronta pomocí dvou zásobníků
- `manual_queue2Stacks.js` – ruční režim algoritmu Fronta pomocí dvou zásobníků
- `actions_queue2Stacks.js` – operace algoritmu Fronta pomocí dvou zásobníků
- `syntax_queue2Stacks.js` – syntaxe algoritmu Fronta pomocí dvou zásobníků
- `random_queue2Stacks.js` – náhodný režim algoritmu Fronta pomocí dvou zásobníků
- `specialCases_queue2Stacks.js` – speciální případy algoritmu Fronta pomocí dvou zásobníků
- `state_splayTree.js` – globální stav algoritmu Splay strom
- `ui_splayTree.js` – rozhraní algoritmu Splay strom
- `manual_splayTree.js` – ruční režim algoritmu Splay strom
- `actions_splayTree.js` – operace algoritmu Splay strom
- `syntax_splayTree.js` – syntaxe algoritmu Splay strom
- `random_splayTree.js` – náhodný režim algoritmu Splay strom
- `specialCases_splayTree.js` – speciální případy algoritmu Splay strom

### 4.2.2 Hlavní části aplikační logiky

Velmi důležitou součástí implementace je hlavní aplikační logika, která zajišťuje propojení jednotlivých částí celé aplikace. Tato vrstva se stará zejména o správu globálního stavu, změny zobrazeného obsahu, aktualizaci rozhraní a návaznost mezi jednotlivými moduly. Právě díky této části aplikace spolu mohou správně spolupracovat sdílené prvky, jednotlivé algoritmy i různé režimy spuštění.

Jako ukázka hlavní aplikační logiky byla vybrána funkce `changeLanguage()`. Tato funkce se nachází v souboru `language_app.js`. Její výběr je vhodný proto, že dobře ukazuje centrální řízení aplikace, protože při změně jazyka neaktualizuje pouze texty v rozhraní, ale také pracuje s uloženým nastavením, mění jazyk dokumentu, obnovuje aktuální zobrazení aplikace a zajišťuje správnou aktualizaci otevřených modálních oken i dalších aktivních prvků.

```
function changeLanguage(language){
  closeNavbarMenu();
  localStorage.setItem('language', language);
  document.documentElement.lang = language === 'cz' ? 'cs' : 'en';

  const Lall = trAll(language);

  langAppApplyCommonStaticTexts(Lall);
  langAppApplyAccessibilityTexts(Lall);
  langAppApplyAlgorithmSelectionTexts(Lall);
  langAppApplyPageTexts(language, Lall);
  langAppApplyParameterButtonTexts(Lall);
  langAppApplyManualModalTexts(Lall, language);
  langAppApplyRandomModalTexts(language, Lall);
  langAppApplySyntaxModalTexts(Lall);
  langAppApplySpecialCasesModalTexts(Lall);

  refreshCurrentViewAfterLanguageChange();
  langAppRefreshRandomOperationPreviews();

  if(typeof refreshOpenTransientModalsForLanguage === 'function')
    refreshOpenTransientModalsForLanguage();
  if(typeof refreshOpenSpecialCasesModalForLanguage === 'function')
    refreshOpenSpecialCasesModalForLanguage();
  if(typeof refreshOpenHistoryModalForLanguage === 'function')
    refreshOpenHistoryModalForLanguage();
  if(typeof updateThemeToggleUI === 'function')
    updateThemeToggleUI();
}
```

Listing 4.1: Ukázka hlavní aplikační logiky ve funkci changeLanguage()

Na uvedené ukázce 4.1 je vidět, že hlavní aplikační logika není omezena jen na jednu dílčí činnost, ale propojuje více částí systému současně. Funkce nejprve uloží zvolený jazyk, následně upraví jazyk dokumentu a poté aktualizuje textové prvky celého rozhraní. Současně zajistí obnovení právě zobrazené části aplikace a případně i aktualizaci otevřených modálních oken nebo dalších aktivních prvků. Tento způsob řešení podporuje centralizaci důležité logiky na jednom místě a zároveň usnadňuje správu i další rozšiřování aplikace.

### 4.2.3 Implementace vybraných algoritmů

V praktické části byly implementovány tři algoritmy: *Multipop on Stack*, *Queue using Two Stacks* a *Splay Tree*. Každý z nich reprezentuje jiný typ chování vhodný pro amortizovanou analýzu. První algoritmus pracuje se zásobníkem založeným na principu LIFO, druhý simuluje frontu s principem FIFO pomocí dvou zásobníků a třetí algoritmus vychází z vlastností binárního vyhledávacího stromu BST, který se při operacích dynamicky přizpůsobuje. Z důvodu přehlednosti nejsou v této podkapitole uvedeny celé implementace algoritmů, ale pouze vybrané a zkrácené ukázky kódu, na kterých je dobře vidět hlavní princip jejich realizace.

Jako první ukázka byla vybrána část implementace algoritmu *Multipop on Stack*, konkrétně operace `multipopFromStackManual(count)` ze souboru `manual_multipop.js`. Tato ukázka byla zvolena proto, že přímo vystihuje hlavní princip algoritmu, tedy odebrání více prvků ze zásobníku najednou, přičemž skutečný počet odebraných prvků závisí na aktuálním stavu zásobníku.

```
function multipopFromStackManual(count){
  if(mpBusy)
    return null;

  const beforeState = mpCreateStateSnapshot();
  const actualCount = Math.min(count, beforeState.size);

  if(actualCount === 0)
    return [];

  const removedValues = beforeState.snapshot.slice(-actualCount);
  const afterSnapshot = beforeState.snapshot.slice(0, beforeState.size -
      actualCount);

  mpApplyManualStateAndRefresh(
    afterSnapshot,
    beforeState.stepCount + actualCount,
    afterSnapshot.length
  );

  return removedValues;
}
```

Listing 4.2: Zkrácená ukázka implementace operace multipop

Na ukázce 4.2 je vidět, že implementace nejprve zjistí aktuální stav zásobníku a následně určí skutečný počet prvků, které lze odebrat. Ten je omezen minimem mezi požadovaným počtem a aktuální velikostí zásobníku. Právě tato vlastnost dobře vystihuje rozdíl mezi požadovanou a skutečně provedenou prací, což je pro tento algoritmus z hlediska amortizované analýzy podstatné.

Jako druhá ukázka byla vybrána část implementace algoritmu *Queue using Two Stacks*. Konkrétně jde o funkci `qTransferAllFromInToOut(done)` ze souboru `manual_queue2Stacks.js`. Tato ukázka byla zvolena proto, že zachycuje klíčový mechanismus celé struktury, tedy přesun prvků ze vstupního zásobníku do výstupního zásobníku.

```
function qTransferAllFromInToOut(done){
  if(qIn.length === 0){
    qPotential = qIn.length;
    updateQueueCounters();

    if(done)
      done();
    return;
  }

  const value = qIn[qIn.length - 1];
  qAnimateRemoveTopItem('qInView', () => {
    qIn.pop();
    qStepCount += 1;

    qOut.push(value);
    qStepCount += 1;
    qPotential = qIn.length;

    renderQueueStacks();
    qAnimateAddTopItem('qOutView');
    updateQueueCounters();

    qRunTimer = setTimeout(() => {
      qTransferAllFromInToOut(done);
    }, qSkipAnimations ? 0 : 250);
  });
}
```

Listing 4.3: Zkrácená ukázka přesunu prvků při implementaci fronty pomocí dvou zásobníků

Ukázka 4.3 vystihuje princip, na kterém je založena realizace fronty pomocí dvou zásobníků. Pokud je výstupní zásobník prázdný, prvky se postupně přesouvají ze vstupního zásobníku, čímž dojde k obrácení jejich pořadí. Právě tímto způsobem je možné zajistit chování odpovídající principu FIFO, přestože vnitřně aplikace pracuje se dvěma zásobníky.

Jako třetí ukázka byla vybrána část implementace algoritmu *Splay Tree*, konkrétně funkce `sRotateLeft(node)` ze souboru `manual_splayTree.js`. Tato ukázka byla zvolena proto, že rotace představují základní mechanismus, pomocí kterého se splay strom při operacích strukturálně upravuje.

```
function sRotateLeft(node){
  const rightChild = node.right;
  if(!rightChild)
    return;

  sStepCount += 1;

  node.right = rightChild.left;
  if(rightChild.left)
    rightChild.left.parent = node;

  rightChild.parent = node.parent;

  if(!node.parent)
    sRoot = rightChild;
  else if(node === node.parent.left)
    node.parent.left = rightChild;
  else
    node.parent.right = rightChild;

  rightChild.left = node;
  node.parent = rightChild;
}
```

Listing 4.4: Zkrácená ukázka levé rotace ve splay stromu

Na ukázce 4.4 je vidět, jak se při levé rotaci mění vazby mezi vrcholy stromu. Přestože jde jen o zkrácenou část implementace, dobře ukazuje, že splay strom není založen pouze na běžném vyhledávání ve struktuře BST, ale také na dynamické změně jejího tvaru pomocí rotací. Právě tyto strukturální úpravy jsou pro chování splay stromu a jeho amortizovanou analýzu zásadní.

### 4.2.4 Zpracování vstupů a režimů spuštění

Důležitou součástí implementace je také zpracování vstupů a jednotlivých režimů spuštění aplikace. Uživatel může pracovat s několika způsoby zadání, konkrétně ručním vstupem, náhodně generovaným vstupem, vstupem pomocí syntaxe a připravenými speciálními případy. Pro přehlednost a znovupoužitelnost byla tato část řešena pomocí společné vrstvy `typesParams`, která sjednocuje obsluhu jednotlivých typů vstupů pro všechny implementované algoritmy.

Jako ukázka zpracování vstupů a režimů spuštění byla vybrána zkrácená část konfigurace. Konkrétně jde o `ALG_REGISTRY`. Ukázka přehledně zachycuje, jak jsou pro jednotlivé algoritmy definovány dostupné režimy spuštění. Současně je na ní vidět, jak je nad těmito režimy vystavěna společná obslužná vrstva. Z ukázky je také patrné, že aplikace neřeší každý algoritmus zcela samostatně, ale používá jednotný princip pro ruční vstup, náhodné generování, speciální případy i syntax režim.

```
const ALG_REGISTRY = Object.freeze({
  multipop: {
    contexts: {manual: 'multipopManual', random: 'multipopRandom', specialCases: 'multipopSpecialCases', syntax
        : 'multipopSyntax'},
    manual: {type: 'stackModal', start: typesParamsStartMultipopWithValues},
    random: {type: 'randomModal', start: typesParamsStartMultipopRandomMode},
    specialCases: {getScenarios: typesParamsGetMultipopSpecialScenarios, start:
        typesParamsRunMultipopSpecialScenario},
    syntax: {resetBeforeOpen: typesParamsResetMultipopSyntaxState, start: typesParamsStartMultipopSyntax}
  },
  queue2Stacks: {
    contexts: {manual: 'queueManual', random: 'queueRandom', specialCases: 'queueSpecialCases', syntax: '
        queueSyntax'},
    manual: {type: 'stackModal', start: typesParamsStartQueueWithValues},
    random: {type: 'randomModal', start: typesParamsStartQueueRandomMode},
    specialCases: {getScenarios: typesParamsGetQueueSpecialScenarios, start: typesParamsRunQueueSpecialScenario
        },
    syntax: {resetBeforeOpen: typesParamsResetQueueSyntaxState, start: typesParamsStartQueueSyntax}
  },
  splayTree: {
    contexts: {manual: 'splayManual', random: 'splayRandom', specialCases: 'splaySpecialCases', syntax: '
        splaySyntax'},
    manual: {type: 'actionInput', titleKey: 'splayInitTitle', labelKey: 'splayInitLabel', infoKey: '
        splayInitInfo', invalidKey: 'splayInitInvalid', start: typesParamsStartSplayManualWithValues},
    random: {type: 'randomModal', start: typesParamsStartSplayRandomMode},
    specialCases: {getScenarios: typesParamsGetSplaySpecialScenarios, start: typesParamsRunSplaySpecialScenario
        },
    syntax: {start: typesParamsStartSplaySyntax}
  }
});
```

Listing 4.5: Zkrácená ukázka konfigurace vstupních režimů

Na ukázce 4.5 je vidět, že vrstva `typesParams` sjednocuje obsluhu všech hlavních režimů spuštění aplikace. Pro každý algoritmus jsou zde definovány kontexty jednotlivých režimů, tedy manuální vstup, náhodný vstup, speciální případy a syntax režim. Současně je zde určeno, jaký typ vstupního

rozhraní se má použít a která funkce zajistí samotné spuštění zvoleného režimu. Toto řešení přispívá k větší modularitě aplikace, protože společná logika zpracování vstupů není duplikována v každém algoritmu zvlášť. Vrstva `typesParams` tak tvoří rozhraní mezi uživatelským zadáním a konkrétní implementací jednotlivých algoritmů.

## 4.3 Uživatelské rozhraní a responzivita

UI je důležitou částí celé aplikace, protože zprostředkovává práci s algoritmy, vstupy i jednotlivými režimy spuštění. Současně bylo nutné navrhnout jej tak, aby zůstalo přehledné při dynamických změnách obsahu a bylo použitelné i na různých typech zařízení.

### 4.3.1 Struktura uživatelského rozhraní a práce s dynamickým obsahem

Součástí implementace je návrh struktury UI a způsob, jakým je do stránky vkládán a měněn dynamický obsah. Aplikace není vytvořena jako sada více samostatných statických stránek, ale jako jedna hlavní dynamická stránka, ve které se podle zvoleného algoritmu a režimu spuštění mění zobrazovaný obsah přímo v rámci DOM.

Jako ukázka byla vybrána zkrácená část HTML struktury z těla dokumentu. Tato ukázka pochází ze souboru `index.html`. Přehledně zachycuje hlavní navigaci, výběr algoritmů, prostor pro dynamicky vkládaný obsah, modální okna i další prvky rozhraní.

```
<body>
  <canvas id="bgCanvas" aria-hidden="true"></canvas>

  <!-- Hlavni navigace -->
  <nav class="navbar ..." >
    ...
    <button id="aboutAppBtn" ...>...</button>
    <div id="backArrow">
      <button id="backBtn" ...>...</button>
    </div>
  </nav>

  <!-- Vyber algoritmu -->
  <div class="alg-panel" id="algorithmLinks">
    ...
    <button id="algorithm1" onclick="changeContent('multipop')">...</button>
    <button id="algorithm2" onclick="changeContent('queue2Stacks')">...</button>
    <button id="algorithm3" onclick="changeContent('splayTree')">...</button>
  </div>

  <!-- Dynamicky obsah -->
  <div id="dynamicContent" class="text-center mt-5"></div>
```

*Pokračování ukázky kódu.*

```
  <!-- Priklad modalniho okna -->
  <div class="modal fade" id="aboutModal" ...>
    <div class="modal-dialog ...">
      <div class="modal-content">
        <div class="modal-header">
          <h5 id="aboutModalLabel"></h5>
        </div>
        <div class="modal-body" id="aboutModalBody"></div>
        <div class="modal-footer">
          <button id="closeBtn" ...></button>
        </div>
      </div>
    </div>
  </div>

  <!-- Dalsi modalni okna -->
  ...
  <footer>
    ...
    <button id="themeToggle" class="theme-toggle"></button>
    <span id="footerText"></span>
  </footer>

  <!-- Nacteni JS souboru -->
  <script src="..."></script>
  ...
</body>
```

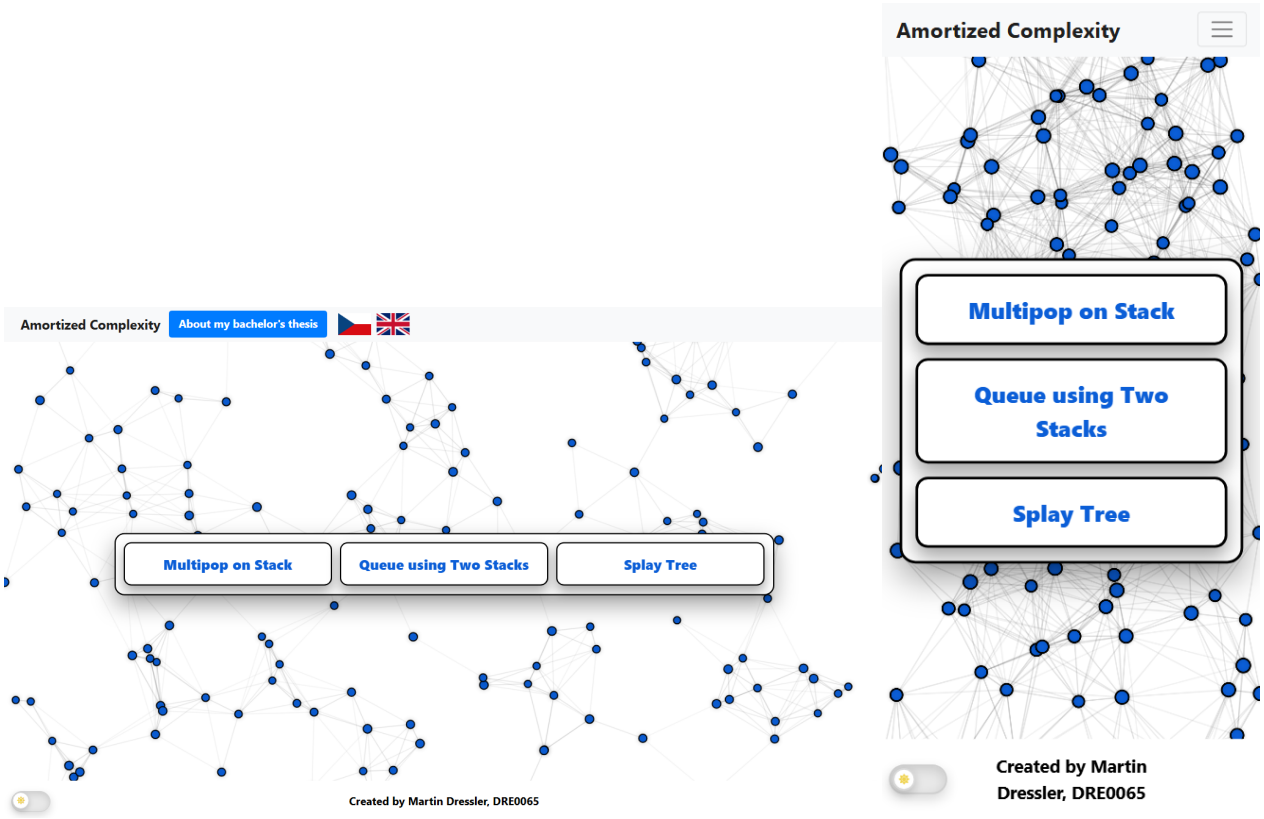
Listing 4.6: Zkrácená ukázka struktury uživatelského rozhraní v souboru index.html

Na ukázce 4.6 je vidět, že uživatelské rozhraní je postaveno na jedné společné HTML kostře. Ta obsahuje hlavní navigaci, úvodní panel pro výběr algoritmů, centrální prvek `dynamicContent` určený pro dynamicky vkládaný obsah, sadu modálních oken a také patičku s dalšími ovládacími prvky. Právě element `dynamicContent` slouží jako hlavní prostor, do kterého aplikace podle aktuální situace vkládá nové části rozhraní. Současně je z ukázky patrné, že důležitou roli hrají i modální okna, která slouží pro zadávání vstupních parametrů, zobrazení doplňujících informací nebo práci s historií kroků. Významná je také návaznost na načítané JS soubory.

### 4.3.2 Responzivní zobrazení na různých zařízeních

V průběhu návrhu celé aplikace byl brán ohled také na responzivní zobrazení na mobilních zařízeních, i když tato část nebyla přímo požadována v zadání. Cílem bylo, aby aplikace zůstala použitelná a přehledná nejen na běžném monitoru, ale i na menších obrazovkách.

Na obrázku 4.3.1 je zachyceno porovnání hlavní stránky na desktopovém a mobilním zařízení. Ze srovnání je patrné, že aplikace si zachovává stejné základní prvky rozhraní, ale jejich rozmístění a velikost se přizpůsobují dostupnému prostoru obrazovky.




(a) Hlavní stránka aplikace na desktopovém zařízení

(b) Hlavní stránka aplikace na mobilním zařízení

Obrázek 4.3.1: Porovnání zobrazení hlavní stránky na různých typech zařízení

Responzivní chování bylo řešeno v souboru `styles.css` pomocí pravidel `@media`.

```
@media(max-width: 767.98px){
  ...
}
```

Listing 4.7: Zkrácená ukázka responzivních pravidel pro mobilní zařízení

Na ukázce 4.7 je vidět, že pro menší obrazovky byla použita samostatná část stylů, která upravuje zobrazení aplikace pro mobilní zařízení. Díky tomu bylo možné zachovat přehlednost rozhraní i při výrazně menším dostupném prostoru.

## 4.4 Nasazení a zpřístupnění aplikace

Po dokončení implementace byla aplikace nasazena a zpřístupněna prostřednictvím školního serveru Homel, který se jevil jako nejvhodnější řešení pro hosting. Díky tomu je možné práci snadno veřejně zpřístupnit v rámci univerzitního prostředí a současně je zajištěn jednoduchý způsob, jak aplikaci prezentovat, testovat a dále využívat.

Výsledná aplikace je dostupná pomocí protokolu HTTP na adrese URL `https://homel.vsb.cz/~dre0065/DRE0065_BP/html/`.

# Kapitola 5

# Závěr

Hlavním cílem této bakalářské práce bylo vytvořit komponentu výukového serveru pro oblast amortizované složitosti, která by budoucím studentům usnadnila pochopení tohoto tématu pomocí názorné a interaktivní vizualizace. V praktické části byla navržena a implementována webová aplikace umožňující práci s vybranými algoritmy vhodnými pro amortizovanou analýzu. Konkrétně byly zpracovány algoritmy Multipop on Stack, Queue using Two Stacks a Splay Tree, u nichž je možné sledovat jednotlivé kroky, změny stavů datových struktur i další hodnoty důležité pro pochopení jejich chování.

Součástí práce bylo také navržení struktury celé aplikace, zpracování různých typů vstupů, vytvoření uživatelského rozhraní a řešení responzivního zobrazení na různých typech zařízení. V teoretické části byly popsány základní pojmy související s amortizovanou složitostí i hlavní části implementace navržené komponenty.

Výsledkem práce je funkční výukový nástroj, který může sloužit jako podpora při výuce i samostatném studiu a který může být v budoucnu rozšířen o další algoritmy. Současně tato práce shrnuje důležité informace o amortizované složitosti, vybraných algoritmech i samotné implementaci komponenty, které jsou popsány v tomto dokumentu ve formátu PDF.

# Zdroje


1. SAWA, Zdeněk. *Teoretická informatika*. Dostupné také z: https://www.cs.vsb.cz/sawa/ti/slides/ti-slides-03.pdf. Podklady pro přednášky, VŠB – Technická univerzita Ostrava.
2. MASSACHUSETTS INSTITUTE OF TECHNOLOGY. *Big O notation*. Dostupné také z: https://web.mit.edu/16.070/www/lecture/big_o.pdf. Studijní materiál.
3. TARJAN, Robert Endre. Amortized Computational Complexity. *SIAM Journal on Algebraic and Discrete Methods*. 1985, roč. 6, č. 2, s. 306–318. Dostupné z DOI: 10.1137/0606031.
4. KOZEN, Dexter. *CS 3110 Lecture 20: Amortized Analysis*. Dostupné také z: http://www.cs.cornell.edu/courses/cs3110/2011sp/lectures/lec20-amortized/amortized.htm. Cornell University.
5. CARNEGIE MELLON UNIVERSITY. *Amortized Analysis*. Dostupné také z: https://www.cs.cmu.edu/~15451-f23/lectures/lecture05-amortized.pdf. Lecture notes for 15-451: Algorithm Design and Analysis.
6. CORNELL UNIVERSITY. *Lecture 25: Amortized Analysis*. 2014. Dostupné také z: https://www.cs.cornell.edu/courses/cs3110/2014fa/lectures/25/lec25.pdf. CS 3110 lecture notes.
7. PUCELLA, Riccardo. *Notes on Programming Standard ML of New Jersey*. 2001. Dostupné také z: https://www.cs.cornell.edu/riccardo/prog-smlnj/notes-011001.pdf. Lecture notes.
8. SLEATOR, Daniel Dominic; TARJAN, Robert Endre. Self-Adjusting Binary Search Trees. *Journal of the ACM*. 1985, roč. 32, č. 3, s. 652–686. Dostupné také z: https://www.cs.cmu.edu/~sleator/papers/self-adjusting.pdf.